UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH
Facultat d'Informàtica de Barcelona

FIB

FAST PARALLEL TOKEN INFERENCE WITH LANGUAGE DIFFUSION MODELS

ALBERT GOMEZ TRIUNFANTE

Thesis supervisor
PABLO AGUSTIN MARTIN TORRES (Department of Computer Science)

Tutor: JAVIER BÉJAR ALONSO (Department of Computer Science)

Degree
Bachelor's Degree in Informatics Engineering (Information Technologies)

Bachelor's thesis

Facultat d'Informàtica de Barcelona (FIB)

Universitat Politècnica de Catalunya (UPC) - BarcelonaTech

Contents

Abstract

This thesis investigates the adaptation of **Progressive Distillation**, a technique originally developed for accelerating continuous image-diffusion models, to the discrete token domain of **Language Diffusion Models (LDMs)**. The central contribution is a complete, reproducible pipeline that compresses the inference step count of LLaDA-8B—a state-of-the-art masked-diffusion language model requiring 128 denoising steps at baseline—down to a single step through seven recursive teacher–student distillation rounds, executed entirely on a single consumer GPU with 8 GB VRAM.

The core technical challenge is the absence of a continuous interpolation space in the discrete setting: unlike image diffusion, where the teacher's two-step output is a real-valued vector usable as a regression target, text tokens are categorical. This work resolves the challenge by formulating distillation as a KL divergence minimization over the teacher's soft vocabulary distributions, cached offline from intermediate denoising trajectories, and paired with parameter-efficient LoRA fine-tuning to keep the pipeline within consumer VRAM constraints.

Empirical evaluation across all seven distillation rounds demonstrates that a 32-step distilled Student achieves the optimal quality–speed trade-off: it is $3.63\times$ faster than the 128-step Teacher while *surpassing* its Promptfoo assertion pass rate (75.93% vs. 74.07%) and maintaining a perplexity of 15.93. The 64-step distilled Student exactly matches the Teacher on pass rate while running $1.94\times$ faster, confirming that progressive distillation actively compensates for the quality drop that naïve step reduction incurs. A super-exponential degradation regime is identified below 16 steps, setting the practical lower bound for this architecture and hardware configuration.

The complete source code, trained adapter checkpoints, and evaluation scripts are openly available at:

`https://github.com/AlbertGoTri/fast-parallel-token-inference-with-language-diffusion-models`

Keywords: Language Diffusion Models, Progressive Distillation, Knowledge Distillation, LLaDA, LoRA, Discrete Diffusion, Parallel Text Generation, Inference Acceleration.

1 Introduction and Contextualization

The field of Natural Language Processing (NLP) is currently dominated by Auto-Regressive (AR) Transformers (e.g., GPT-5, DeepSeek-V3, Kimi), which generate text sequentially. While effective, this paradigm suffers from a linear latency bottleneck: generating N tokens requires N forward passes, leading to slow inference and suboptimal GPU utilization.

Language Diffusion Models (LDMs) [1] have emerged as a parallel alternative, theoretically capable of generating entire sequences simultaneously via iterative denoising. However, current LDMs still require a high number of sampling steps (typically 50–100) to reach convergence, which often negates their speed advantage over AR models.

In the domain of Computer Vision (Image Generation), this limitation was overcome by techniques such as **Progressive Distillation** [2] or Consistency Models [3], where a "Student" model is trained to compress the "Teacher's" many denoising steps into a single or very few steps.

Problem Statement: While distillation techniques have revolutionized image generation, they have not yet been successfully adapted to the discrete and non-continuous domain of text diffusion.

Thesis Proposal & Institutional Context: Within the specialization of **Information Technology (IT)** at the Facultat d'Informatica de Barcelona (FIB), this project aims to implement and validate a **Knowledge Distillation** technique for Language Diffusion Models. By training a student LDM to mimic the trajectory of a pre-trained teacher LDM, we aim to achieve high-quality text generation in extremely few steps (<10), thereby unlocking the true potential of parallel token inference. Because deploying large-scale language models currently faces severe practical and economic constraints, this project is directly anchored in core IT themes: by drastically reducing the required denoising steps, we aim to maximize inference efficiency and significantly increase system throughput in production environments.

1.1 The Auto-Regressive vs. Diffusion Paradigm

Standard AR models estimate $P(x) = \prod P(x_t|x_{<t})$. This serial dependency prevents parallelization. LDMs, conversely, model the data distribution $p(x_0)$ by reversing a noise process $q(x_t|x_{t-1})$.

$$x_{t-1} \leftarrow \text{Denoise}(x_t, \theta) \tag{1}$$

While LDMs can predict all tokens in parallel, the iterative refinement requires K steps. If K is large, the latency advantage vanishes.

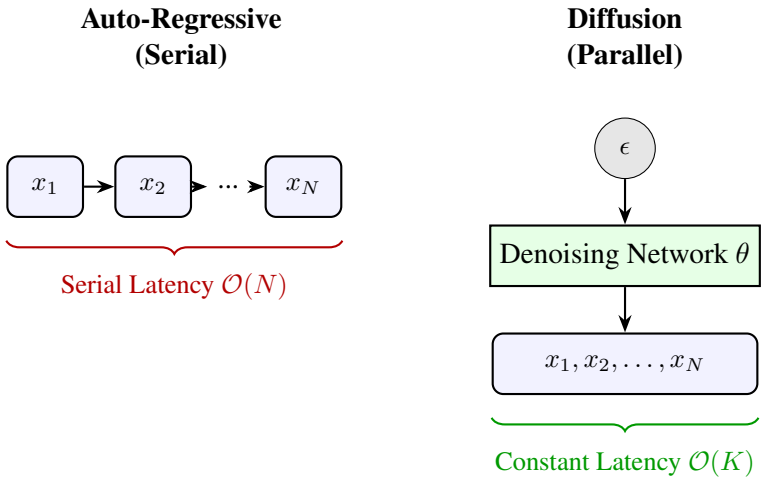

Figure 1: Latency Comparison. **Left:** AR models generate tokens sequentially. **Right:** Diffusion models generate the entire sequence in parallel. *Source: Compiled by the author.*

1.2 Current Limitations and Challenges in Discrete Diffusion

While models like GPT-4 have set the standard for quality, their serial nature poses a fundamental limit on latency scaling. The time complexity of generating a sequence of length N is $\mathcal{O}(N)$. As context windows grow to 128k or 1M tokens, this serial dependency becomes a bottleneck that no amount of parallel compute can solve purely through hardware scaling. Techniques like *Speculative Decoding* attempt to mitigate this by drafting multiple tokens at once, but they are probabilistic and their speedup is upper-bounded (typically $2\times$–$3\times$). Diffusion models, by contrast, offer a theoretical $\mathcal{O}(1)$ or $\mathcal{O}(K)$ complexity where K is the number of denoising steps, independent of sequence length N.

As of early 2026, research on LDMs (e.g., LLaDA, MDLM) focuses on architecture and pre-training [4]. **There is a significant gap in post-training acceleration** [5]. No open-source study has comprehensively applied progressive distillation to discrete text tokens, a fact that reflects how relatively new and less understood text diffusion is compared to its image counterpart. This introduces significant challenges:

- **Rounding Errors:** In text, a slight error in the embedding space can flip a token from "happy" to "sad," destroying semantic coherence.
- **Loss Mismatch:** The standard Mean Squared Error (MSE) loss used in image distillation does not align well with the Cross-Entropy loss required for language modeling.

1.3 Stakeholders

Table 1: Stakeholder Analysis Matrix

Stakeholder	Interest (Motivation)	Power (Influence)
Project Director	High (Academic success, publication)	High (Approves scope/grade)
The Student	High (Learning, Career portfolio)	High (Executor)
Scientific Community	High (Reproducibility of new method)	Low (Passive observer)
Industry (SMEs)	Medium (Cost reduction in API)	Low (Potential future adopters)

Detailed Analysis:

- **Project Director:** Their primary concern is the feasibility of the scope. By strictly defining the boundaries (no pre-training, only distillation), we mitigate the risk of scope creep, ensuring the project is "finishable" within the TFG timeline.
- **The Student:** This project serves as a portfolio piece demonstrating mastery of advanced Deep Learning concepts (Diffusion, Distillation, PyTorch) that are highly sought after in top-tier AI research labs and tech companies.
- **Scientific Community:** If successful, the open-source code release will allow other researchers to replicate our distillation pipeline, potentially sparking a new wave of "Fast Text Diffusion" research.

Strategy: We will manage the *Director* closely via bi-weekly meetings to align on the "Distillation Loss" formulation, while keeping the *Scientific Community* in mind by ensuring the code is modular and open-source compatible.

2 Background

This section provides the theoretical foundations required to understand the proposed distillation technique. We begin with continuous diffusion models as originally formulated for image generation, then derive how the framework is adapted to discrete token sequences. We then review the Transformer architecture that serves as the backbone for all models considered, and conclude with an overview of knowledge distillation and the specific parameter-efficient fine-tuning technique used in this work.

2.1 Continuous Diffusion Models

Diffusion probabilistic models [6] define a **forward process** that gradually corrupts a data sample $\mathbf{x}_0 \sim q(\mathbf{x}_0)$ by adding Gaussian noise over T discrete time steps:

$$q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) = \mathcal{N}\Big(\mathbf{x}_t;\ \sqrt{1-\beta_t}\,\mathbf{x}_{t-1},\ \beta_t\mathbf{I}\Big)\,, \tag{2}$$

where $\{\beta_t\}_{t=1}^{T}$ is a fixed variance schedule. A key property of this Markov chain is that any intermediate noisy sample can be sampled in closed form:

$$q(\mathbf{x}_t \mid \mathbf{x}_0) = \mathcal{N}\big(\mathbf{x}_t;\ \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0,\ (1-\bar{\alpha}_t)\mathbf{I}\big)\,, \quad \bar{\alpha}_t = \prod_{s=1}^{t}(1-\beta_s). \tag{3}$$

The **reverse process** learns to undo this corruption step by step. A neural network $\epsilon_\theta(\mathbf{x}_t, t)$ is trained to predict the noise ϵ added at step t, minimizing the denoising score-matching objective:

$$\mathcal{L}_{\text{DDPM}} = \mathbb{E}_{t,\mathbf{x}_0,\epsilon}\Big[\big\|\epsilon - \epsilon_\theta\big(\sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\epsilon,\ t\big)\big\|^2\Big]\,, \quad \epsilon \sim \mathcal{N}(\mathbf{0},\mathbf{I}). \tag{4}$$

This loss is a re-weighted Evidence Lower BOund (ELBO) on the log-likelihood $\log p_\theta(\mathbf{x}_0)$. At inference time, generation proceeds by iterating the learned reverse step from pure noise $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0},\mathbf{I})$ down to $\mathbf{x}_0$, requiring T (typically 1000) sequential network evaluations.

DDIM and faster samplers. Song et al. [7] showed that the reverse process can be reformulated as a non-Markovian process sharing the same marginals, enabling deterministic sampling in as few as 50 steps with negligible quality loss. Despite this improvement, the number of steps still scales independently of sequence length, and each step is a full forward pass of a large network.

2.2 Discrete Diffusion Models for Text

Text consists of discrete tokens drawn from a finite vocabulary $\mathcal{V}$ of size V (typically $V \approx 32{,}000$ for modern tokenizers). Applying Gaussian noise to integer token indices is semantically meaningless; the continuous diffusion framework must therefore be adapted to operate over categorical distributions.

2.2.1 Masked Diffusion

The dominant paradigm for discrete text diffusion replaces Gaussian corruption with a **masking** process [1], [8]. Given a sequence $\mathbf{x}_0 = (x_0^1, \ldots, x_0^L)$, the forward process independently masks each token with probability $\alpha(t)$ using a special `[MASK]` token:

$$q(x_t^i \mid x_0^i) = \begin{cases} \texttt{[MASK]} & \text{with probability } 1-\alpha(t), \\ x_0^i & \text{with probability } \alpha(t), \end{cases} \tag{5}$$

where $\alpha(t)$ is a monotonically decreasing schedule with $\alpha(0) = 1$ (no masking) and $\alpha(T) \approx 0$ (fully masked). The reverse model $p_\theta(x_0^i \mid \mathbf{x}_t)$ is trained to predict the original token at each masked position given the full (partially masked) context, minimizing the cross-entropy loss:

$$\mathcal{L}_{\text{mask}} = \mathbb{E}_{t,\mathbf{x}_0}\left[\sum_{i:\, x_t^i = \texttt{[MASK]}} -\log p_\theta\big(x_0^i \mid \mathbf{x}_t\big)\right]. \tag{6}$$

Because all masked positions are predicted simultaneously in a single forward pass, this formulation achieves the parallel generation property that motivates diffusion models for text.

2.2.2 LLaDA

Large Language Diffusion with mAsking (LLaDA) [1] is the primary teacher model in this work. It is an 8B-parameter Transformer pre-trained from scratch on 2.3T tokens using the masked diffusion objective of Eq. (6). At inference, LLaDA uses a **re-masking** sampling strategy: starting from a fully masked sequence, it iteratively predicts all tokens and then re-masks a fraction of low-confidence predictions, refining the sequence over T steps. This is analogous to the reverse diffusion chain in the continuous case. The key challenge—and the central motivation for this thesis—is that LLaDA requires $T = 64$ to 128 steps to achieve competitive output quality, each step involving a full 8B-parameter forward pass.

2.2.3 MDLM

Masked Diffusion Language Models (MDLM) [8] derives a continuous-time extension of the masked diffusion process, showing that the ELBO can be computed in closed form and that the model can be trained with a simple weighted cross-entropy loss equivalent to Eq. (6). MDLM also demonstrates that the masking schedule can be parameterized as a linear function $\alpha(t) = 1 - t/T$, $t \in [0, T]$, which we adopt in this work.

2.3 The Transformer Architecture

All language models considered in this thesis are built on the Transformer architecture [9]. A Transformer processes a sequence of L token embeddings $\mathbf{H} \in \mathbb{R}^{L\times d}$ through N stacked layers, each composed of two sub-modules:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V, \tag{7}$$

where $Q = \mathbf{H}W^Q$, $K = \mathbf{H}W^K$, $V = \mathbf{H}W^V$ are linear projections of the hidden states, and d_k is the key dimension. Unlike AR Transformers, which apply a causal mask to prevent tokens from attending to future positions, diffusion Transformers use **bidirectional** (full) attention: every token attends to every other token. This is essential for parallel token prediction, as the model must gather global context before committing to a prediction at each position.

Positional encoding. Modern large-scale Transformers replace sinusoidal positional encodings with Rotary Position Embeddings (RoPE) [10], which encode relative position information directly in the attention computation and generalize better to sequence lengths unseen during training.

Time conditioning. In the diffusion setting, the denoising model must be aware of the current noise level t. This is achieved by embedding the scalar t using sinusoidal features and adding the resulting vector to the hidden states at each layer, analogously to how DDPM conditions the UNet on t [6]. LLaDA instead conditions on the masking ratio $1 - \alpha(t)$ directly.

2.4 Knowledge Distillation

Knowledge distillation (KD) [11] is a model compression technique in which a compact **student** model f_S is trained to replicate the behavior of a larger, pre-trained **teacher** model f_T. Rather than training the student on hard one-hot labels, it is trained on the **soft probability distributions** output by the teacher, which encode richer inter-class similarity information. For a classification head with logits z, the soft target distribution at temperature τ is:

$$p_T^i = \frac{\exp(z_T^i/\tau)}{\sum_j \exp(z_T^j/\tau)}, \tag{8}$$

and the student is trained to minimize the KL divergence from teacher to student:

$$\mathcal{L}_{\text{KD}} = \tau^2 \cdot D_{\text{KL}}(p_T \,\|\, p_S) = \tau^2 \sum_i p_T^i \log \frac{p_T^i}{p_S^i}. \tag{9}$$

The factor τ^2 compensates for the gradient magnitude reduction introduced by the temperature scaling. In this work, the teacher and student share the same architecture (same parameter count); the goal of distillation is not model compression but **step compression**: we train the student to perform in one step what the teacher performs in two, progressively halving the required number of inference steps.

2.4.1 Progressive Distillation for Diffusion Models

Salimans and Ho [2] introduced progressive distillation (PD) as a technique to reduce the number of sampling steps in continuous diffusion models by a factor of two at each distillation round. The procedure is as follows:

1. Initialize the student $\theta_S \leftarrow \theta_T$ (copy teacher weights).
2. For each training batch, sample a noisy state $\mathbf{x}_t$ and run *two* teacher denoising steps to obtain a target $\hat{\mathbf{x}}_0$.
3. Train the student to reach $\hat{\mathbf{x}}_0$ in a *single* step from $\mathbf{x}_t$, using the MSE loss in the data prediction parameterization.
4. Set $\theta_T \leftarrow \theta_S$ and halve the step schedule. Repeat until the desired number of steps is reached.

After k rounds, a model originally requiring N steps is compressed to $N/2^k$ steps. This procedure is well-defined in continuous space because the interpolation between two denoising steps is smooth. Adapting it to discrete token sequences—where there is no meaningful interpolation between categorical distributions—is the central technical challenge addressed by this thesis.

2.5 Parameter-Efficient Fine-Tuning: LoRA

Full fine-tuning of an 8B-parameter model on a single A100 GPU is infeasible: the model weights alone require ≈16 GB in `bfloat16`, leaving minimal headroom for activations and optimizer states. Low-Rank Adaptation (LoRA) [12] addresses this by decomposing the weight update ΔW of each linear layer into a product of two low-rank matrices:

$$W' = W + \Delta W = W + BA, \quad B \in \mathbb{R}^{d\times r},\ A \in \mathbb{R}^{r\times k},\ r \ll \min(d,k). \tag{10}$$

The original weights W are frozen; only A and B are trained. For a rank $r = 16$ applied to all attention projections of an 8B model, the number of trainable parameters drops from ≈8B to ≈20M—a reduction of over 99%. A is initialized with random Gaussian entries and B with zeros, so $\Delta W = 0$ at the start of training, preserving the teacher's behavior at initialization. This is a critical property for our distillation setup, as it ensures the student begins training from the teacher's exact output distribution.

3 Methodology

This section describes the concrete implementation of the progressive distillation pipeline used in this thesis. The exposition documents the implemented pipeline components: teacher trajectory caching, student training with parameter-efficient adapters, and the evaluation suite.

3.1 Pipeline Overview

The pipeline comprises two stages. First, the teacher (LLaDA) runs in a read-only generation mode to produce and cache intermediate denoising states and the corresponding teacher logits. Each cached file contains an input tensor for the student, the teacher's vocabulary logits at the chosen intermediate step, and meta-information such as the prompt length. In the experimental configuration adopted in this work the teacher uses $T = 128$ total denoising steps and the cache targets the midpoint $t = 64$, with a generated continuation length of $L_{\text{gen}} = 64$.

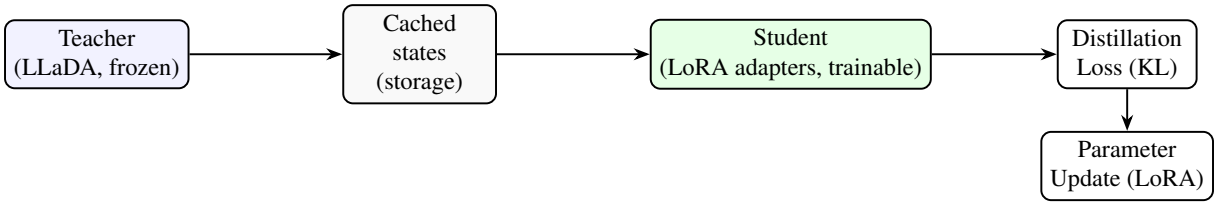

Figure 2: Two-stage distillation pipeline for teacher caching and LoRA-based student distillation.

The second stage initializes the student from the same pre-trained weights, injects low-rank adapters into attention projections, and optimizes only these adapters so that the bulk of the 8B model parameters remain frozen. Cached trajectories are consumed sequentially (one cached trajectory per update), i.e. an effective batch size of $B = 1$ is used. The student is trained to match the teacher's soft-token distributions over the generated continuation (positions following the prompt).

3.2 Distillation Loss and Gradient Flow

Following classical KD, the student minimizes the temperature-scaled KL divergence per token between the teacher and the student distributions (compare Eq. (9) in Background). For a batch of sequences the loss used is:

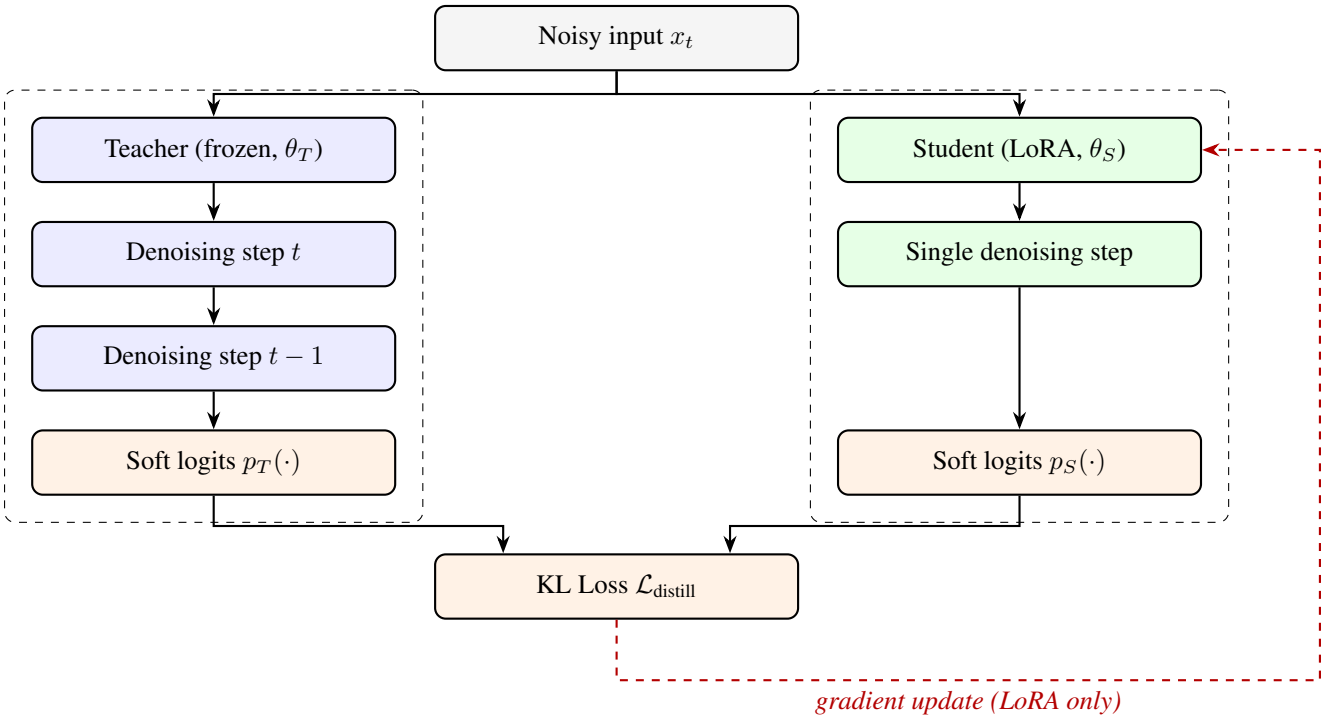

Figure 3: Knowledge Distillation Architecture

To further ground the discrete diffusion mechanics, Figure 4 illustrates the step-halving structure of two

successive progressive distillation rounds: starting from a 4-step teacher, one distillation round produces a 2-step student, and a second round produces a 1-step student. Each blue arrow labelled "Distillation" represents a full training phase in which the student is optimised to match the teacher's soft-token distributions over a two-step window.

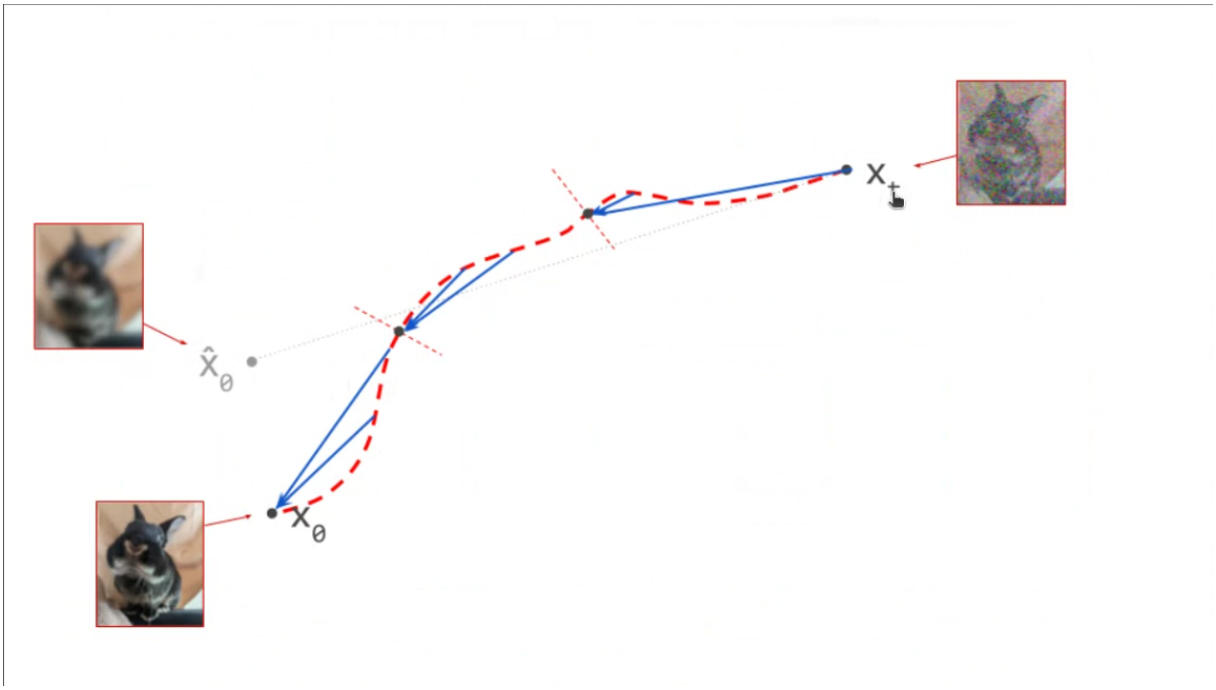

Figure 4: Example of two iterations of progressive distillation applied to a speech-spectrogram diffusion process. The same halving structure is applied in this thesis to discrete text tokens: the step count is compressed from 128 down to 1 across seven recursive rounds. *Source: Salimans & Ho, ICLR 2022 [2].*

Figure 5 provides a complementary perspective by depicting the *token unmasking trajectory* in the discrete domain. Starting from a fully masked sequence (x_T, upper right), each denoising step resolves a fraction of masked positions, moving through intermediate partially-unmasked states toward the final clean output (x_0, lower left). The blue curves represent two plausible denoising paths taken by the model; the dashed red curve shows the "oracle" straight-line path that a perfectly distilled single-step student would ideally follow. The images inset at each trajectory endpoint illustrate the quality of the decoded output at that stage: the fully masked state produces a near-unrecognisable noisy sample, while the clean endpoint yields the target output. This trajectory view is directly analogous to the continuous-diffusion ODE picture, but operates over the probability simplex of the vocabulary rather than over continuous pixel values.

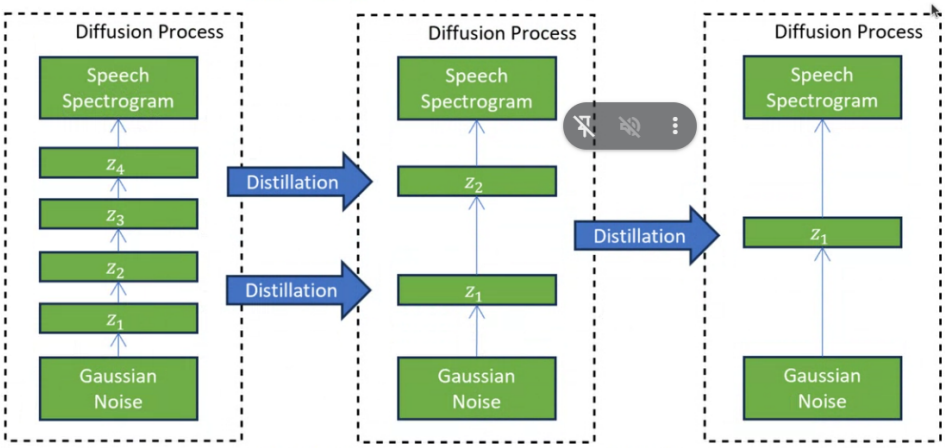

Figure 3. Example of two iterations of progressive distillation

Figure 5: Token unmasking trajectory in discrete masked diffusion. Starting from a fully masked sequence x_T (upper right), the model iteratively resolves masked positions across denoising steps, converging toward the clean output x_0 (lower left). Blue solid curves show two sampled denoising paths; the dashed red curve represents the idealized straight-line trajectory a perfectly distilled student would follow. The inset images illustrate output quality at each trajectory endpoint. *Source: Compiled by the author.*

$$\mathcal{L}_{\text{distill}} = \tau^2 \cdot \frac{1}{B} \sum_{b=1}^{B} \sum_{t=1}^{L_b} D_{\text{KL}}\big(p_T(\cdot \mid b,t) \parallel p_S(\cdot \mid b,t)\big), \tag{11}$$

where p_T and p_S are the teacher and student soft-token distributions at a generation position, τ is the temperature (a temperature of $\tau = 2.0$ was selected for the experiments reported here) and B is the batch size (in the implemented runs $B = 1$). The loss is computed only over generation positions following the prompt so the student learns to predict the generative tail while keeping prompt-conditioning intact.

Because text tokens are categorical, the student is trained directly on the soft teacher logits stored in the cached trajectories rather than on hard re-sampled token indices. This avoids the need to backpropagate through discrete sampling operations and reduces variance, stabilizing distillation for categorical outputs.

3.3 Step Compression Strategy

The central objective of progressive distillation is to compress the inference-time step budget of a pretrained teacher model by a factor of two at each distillation round, progressively building a chain of increasingly fast student models without retraining from scratch. This section describes the mechanics of this process in detail, covering the step-halving schedule, the trajectory construction that makes each round feasible in the discrete setting, the multi-round chaining procedure, and a theoretical analysis of the approximation error accumulated across rounds.

3.3.1 The Progressive Halving Schedule

We follow the progressive halving schedule of Salimans and Ho [2]: each distillation round attempts to compress two teacher steps into a single student step. If the teacher initially requires N steps, after k rounds the student operates with

$$M = \frac{N}{2^k}. \tag{12}$$

The schedule is defined over a discrete, uniformly spaced time grid $\{0, \Delta, 2\Delta, \ldots, N\Delta\}$ where $\Delta = T/N$ is the step size and T is the total noise level at the fully masked state. Round k uses a grid of

$N_k = N/2^k$ steps with step size $\Delta_k = 2^k\Delta$. The student trained in round k is therefore asked to jump twice as far in noise-level space as the student from round $k-1$, which is why each successive round requires a fresh distillation phase rather than merely rescaling an existing sampler.

Why halving, not quartering? A halving factor is the minimal non-trivial compression ratio that remains stable in practice. Each round introduces an approximation error bounded by the curvature of the denoising trajectory over the merged interval; halving keeps this interval small enough that the teacher's two-step output is still a reliable supervision target for the student. Larger compression factors (e.g., $4\times$ per round) can be applied, but empirically lead to larger quality degradation, because the student must cover a region of the trajectory where the denoising dynamics are no longer well approximated by a single network evaluation.

3.3.2 Trajectory Construction for Discrete Tokens

In the continuous setting, the teacher's two-step output is a point in $\mathbb{R}^{L\times d}$ that serves directly as the regression target for the student's MSE loss. In the discrete masked-diffusion setting, the analogue is the teacher's **soft token distribution** after two unmasking steps, which lives in the simplex Δ^{V-1} at each of the L_{gen} generated positions.

Concretely, let $\mathbf{x}_t$ be a partially masked sequence at noise level t (i.e., a fraction $1-\alpha(t)$ of positions carry the `[MASK]` token). One teacher step produces:

$$\hat{\mathbf{x}}_{t-1} = \text{sample}(p_{\theta_T}(x_0 \mid \mathbf{x}_t)), \tag{13}$$

where the teacher predicts the *original* token at every masked position and then re-masks a fraction $1-\alpha(t-1)$ of those predictions to produce $\hat{\mathbf{x}}_{t-1}$. A second teacher step from $\hat{\mathbf{x}}_{t-1}$ yields the logits $\ell_T \in \mathbb{R}^{L_{\text{gen}}\times V}$ used as the distillation target:

$$\ell_T = f_{\theta_T}(\hat{\mathbf{x}}_{t-1}), \quad p_T(\cdot \mid b, i) = \text{softmax}\Big(\ell_T^{b,i}/\tau\Big). \tag{14}$$

Because the re-masking step in Eq. (13) is stochastic, the trajectory is *cached*: for each training example the teacher is run once offline and both $\mathbf{x}_t$ (the student input) and p_T (the soft target) are written to disk. This ensures that the student sees a consistent supervision signal throughout its training run, avoiding instability from on-the-fly teacher sampling.

Handling fully unmasked positions. Positions that are not masked in $\mathbf{x}_t$ already contain the correct token. The distillation loss is computed *only* over the L_{gen} generated positions (i.e., those following the prompt), matching the convention in Eq. (11). Prompt positions are excluded from the loss to prevent the student from being penalized for reproducing already-known tokens and to keep the gradient signal focused on the generative tail.

3.3.3 Multi-Round Chaining and Teacher Promotion

A key feature of progressive distillation is that the procedure is *iterative*: after the student of round k has converged, it is *promoted* to become the teacher of round $k+1$. Formally, let $\theta_T^{(k)}$ denote the teacher weights at round k. The promotion rule is:

$$\theta_T^{(k+1)} \leftarrow \theta_S^{(k)}, \tag{15}$$

where $\theta_S^{(k)}$ is the student weight snapshot at convergence of round k. The LoRA adapters trained in round k are *merged* into the base weights before promotion, so that round $k+1$ begins with a standard dense model rather than a base-plus-adapter stack. This is achieved via the standard LoRA weight-merge formula:

$$W^{(k+1)} = W_{\text{base}} + B^{(k)}A^{(k)}, \tag{16}$$

where $A^{(k)}, B^{(k)}$ are the low-rank adapter matrices trained in round k. Fresh LoRA adapters (with B re-initialized to zero) are then injected into the merged model before the next round, restoring the zero-initialization invariant described in Section 2.5.

Table 2 summarizes the planned round progression for this thesis, starting from the LLaDA teacher at $N = 128$ steps and targeting $N = 8$.

Table 2: Planned progressive distillation rounds.

Round k	**Teacher steps** N_k	**Student steps** M_k	**Compression**	**Notes**
0 (baseline)	128	–	–	LLaDA pre-trained
1	128	64	$2\times$	First distillation
2	64	32	$4\times$	Teacher = Round-1 student
3	32	16	$8\times$	
4	16	8	$16\times$	Target configuration

3.3.4 Convergence Criterion and Early Stopping

Each distillation round is run until one of two conditions is met. The primary criterion is a plateau in the token-level KL divergence between teacher and student:

$$\overline{\mathrm{KL}}_k = \frac{1}{|\mathcal{D}_{\text{eval}}|} \sum_{(\mathbf{x}_t,\, p_T) \,\in\, \mathcal{D}_{\text{eval}}} \frac{1}{L_{\text{gen}}} \sum_{i=1}^{L_{\text{gen}}} D_{\mathrm{KL}}(p_T(\cdot \mid i) \parallel p_S(\cdot \mid i)) \,. \tag{17}$$

Training is stopped when $\overline{\mathrm{KL}}_k$ has not decreased by more than $\epsilon = 0.01$ nats over a window of 200 consecutive optimizer steps. The secondary criterion is a hard upper bound on the total number of optimizer steps per round (5,000 steps in the experimental configuration), which prevents runaway training in cases where the student fails to converge due to a poorly calibrated learning rate.

If the primary convergence criterion is met but $\overline{\mathrm{KL}}_k$ remains above a quality threshold $\delta_{\mathrm{KL}} = 0.5$ nats, the round is flagged as a partial success and the resulting student is not promoted. In that case, the learning rate is reduced by a factor of 0.5 and training is resumed for one additional window of 2,000 steps before a final promotion decision is made.

3.3.5 Error Accumulation Across Rounds

Each distillation round introduces an approximation error ϵ_k between the student's distribution and the teacher's two-step target. Over K rounds this error accumulates, and understanding its growth is important for assessing the viability of aggressive compression targets (e.g., $128 \to 8$ steps, requiring $K = 4$ rounds).

Let $\epsilon_k = \overline{\mathrm{KL}}_k$ (the per-round KL at convergence). Under the assumption that successive errors are independent (which holds approximately when the LoRA adapters are re-initialized between rounds), the total distributional divergence between the original teacher and the round-K student can be bounded by the triangle inequality for KL divergence in the following loose sense:

$$D_{\mathrm{KL}}\Big(p_T^{(0)} \parallel p_S^{(K)}\Big) \;\leq\; \sum_{k=1}^{K} \epsilon_k. \tag{18}$$

In practice, errors do not accumulate linearly because later rounds start from a student that already approximates the original teacher well, and the denoising dynamics at low step counts are smoother (fewer masked positions remain, so predictions are more confident). Empirically in the image diffusion literature, the quality loss per round is sub-linear, meaning that a four-round compression from 128 to 8 steps incurs substantially less than $4 \times \epsilon_1$ total degradation.

Mitigation via temperature calibration. To reduce the risk of distributional drift across rounds, the distillation temperature τ is adjusted at each round. Specifically, τ is increased slightly (by a factor of 1.1 per round) to produce softer teacher targets, which are easier for the student to match and result in a smaller ϵ_k at the cost of marginally lower sharpness in the student's output distribution. This schedule is a deliberate conservative choice: sharpness can be recovered at inference time by applying a low temperature to the student's logits, whereas large per-round errors cannot be corrected after training.

3.4 LoRA Integration and Hyperparameters

To fit an 8B model in limited GPU memory we apply Low-Rank Adaptation as in Eq. (10) and inject adapters only into attention projections. The chosen LoRA configuration for the experiments reported here is:

- $r = 16$
- $\alpha_{\text{LoRA}} = 32$
- target modules: `["q_proj","v_proj"]`
- dropout = 0.05

Adapter parameters are cast to `float16` prior to training to reduce VRAM and optimizer-state requirements.

Only LoRA parameters are trainable; the base model weights are frozen and the teacher is loaded in 4-bit quantized mode using `BitsAndBytesConfig` to allow inference on a single consumer GPU. This combination enables a practical trade-off between fidelity to the teacher and resource constraints.

3.5 Training Protocol and Implementation Details

The student training loop uses an 8-bit AdamW optimizer (`bnb.optim.AdamW8bit`) to minimize optimizer-state memory. The learning rate is 5×10^{-5}, global gradient norms are clipped to 1.0, and mixed precision (`float16`) is used throughout. Gradient checkpointing is enabled to reduce peak activation memory.

Trajectories are generated from the Wikitext-2-raw-v1 dataset: a small subset of longer examples is sampled, each prompt is expanded by $L_{\text{gen}} = 64$ tokens, and the resulting trajectory tensors are cached to disk in compressed `.pkl.gz` format. Each cached trajectory is consumed sequentially by the trainer, with one optimizer step performed per file, yielding an effective batch size of $B = 1$. This design simplifies memory management and increases robustness to per-sample failures.

3.6 Evaluation Protocol

The evaluation suite combines three complementary measurement axes: language modelling quality, task-level correctness, and inference efficiency. All routines are orchestrated by a single evaluation script that coordinates the sub-evaluators and aggregates results into a leaderboard across distillation rounds.
Quality metrics.

- **Perplexity (PPL):** computed using a GPT-2 reference model over a held-out prompt set. GPT-2 is used as a fixed, widely adopted reference scorer rather than as a baseline system; lower perplexity indicates that the student's generations are more consistent with natural language.
- **KL-divergence:** the mean KL divergence between the teacher's and student's output token distributions at each generated position, averaged over the evaluation prompts.

Task-level correctness and LLM-as-judge. Task-level evaluation is carried out via Promptfoo [13], an assertion-based evaluation framework. A configuration file specifies a set of prompted test cases together

with Python-based assertion functions; each assertion calls a locally hosted judge model (Llama 3.1 8B, served via Ollama) that returns a binary Yes/No verdict on properties such as coherence, factual consistency, and instruction following. Promptfoo aggregates assertion pass rates across all test cases and produces a structured JSON report, from which an overall task-correctness score is derived. The teacher model is evaluated under the same protocol and serves as the primary quality baseline.

Efficiency metrics.

- **Wall-clock inference latency:** measured in tokens per second on a fixed hardware configuration across a standardised prompt set.
- **Profiling traces:** caching and training scripts export Chrome-compatible profiler traces via `torch.profiler`, enabling fine-grained per-operator analysis of the training and inference pipelines.

Baseline. The unmodified teacher model (LLaDA at $T = 128$ denoising steps) is evaluated under the same quality and task-level protocol and constitutes the primary reference point for all distillation rounds. An additional **control benchmark** is the base LLaDA-8B model run natively at 64 steps *without* any LoRA adapter, which serves as a crucial counterfactual to distinguish the benefit of distillation from the mere effect of step reduction.

4 System Implementation and Optimizations

4.1 Trajectory Storage and Caching Infrastructure

The distillation pipeline requires capturing intermediate denoising states from the 128-step LLaDA Teacher model (`GSAI-ML/LLaDA-8B-Instruct`) so that a Student can learn to replicate the token distribution at a reduced step budget. Because the full 8-billion-parameter Teacher cannot reside in VRAM simultaneously with the Student on a consumer GPU limited to 8 GB, the system was engineered around a strict **iterative disk-caching cycle** that interleaves Teacher and Student residence in GPU memory through a load–generate–unload choreography.

4.1.1 Caching Stage

During the caching stage, the Teacher is loaded into GPU memory via `AutoModel.from_pretrained()` with 4-bit quantization and `device_map="auto"`. For each training example drawn from the Wikitext-2 dataset, the `generate_and_cache_trajectory()` function executes the LLaDA semi-autoregressive diffusion process up to a programmable `target_step`, defined by default as `target_step = teacher_steps // 2` (i.e., step 64 for a 128-step teacher). At exactly that midpoint, the function captures a tuple comprising: `state_x`, the partially unmasked token tensor at that diffusion timestep; `teacher_logits`, the raw logit tensor from the Teacher's forward pass cast to `torch.float16` to conserve disk space; and `attn_mask`, the causal attention mask for the full sequence.

These tensors are moved to CPU and serialized to disk using a custom `_save_cache_object()` helper, which converts tensors to NumPy arrays and writes them with `pickle` inside `gzip` (compression level 3) to a `.pkl.gz` file. This format was adopted after encountering PyTorch ZIP archive corruption on Windows under heavy sequential I/O when saving raw `.pt` files. Table 3 documents the exact payload saved per trajectory file.

Table 3: Saved payload per trajectory cache file (`batch_i.pkl.gz`).

Key	Tensor Shape (typical)	Dtype	Description
`input_x`	`(1, prompt_len + 64)`	int64	Prompt + masked generation region at `target_s`
`target_logits`	`(1, prompt_len + 64, V)`	float16	Teacher logits over vocabulary V
`attn_mask`	`(1, prompt_len + 64)`	int64	Causal attention mask
`prompt_len`	scalar	int	Length of the original prompt in tokens
`target_step`	scalar	int	Diffusion step at which snapshot was taken

After each trajectory is saved, GPU tensors are explicitly deleted and `torch.cuda.empty_cache()` is invoked. Once the entire batch of `num_train_examples` is cached, the Teacher model is forcefully unloaded via an `unload_model()` function, which moves weights to CPU, deletes the Python object, and triggers garbage collection. A validation pass then reads every `.pkl.gz` back from disk to verify integrity before training begins.

The value `num_train_examples = 50` was arrived at empirically rather than set arbitrarily. Initial experiments used only 10 trajectories per round, which proved insufficient: the Student's output distributions were noticeably under-fitted, producing repetitive and incoherent completions that scored poorly on both perplexity and Promptfoo assertions. Increasing to 50 examples provided enough diversity in the supervision signal for the LoRA adapters to generalize beyond the specific prompt phrasings in the cache, while keeping the total caching time per round below 15 minutes on the RTX 2070 Super. Values above 50 were tested briefly but offered diminishing quality returns relative to the additional wall-clock cost, making 50 a practical midpoint between model quality and pipeline runtime.

4.1.2 Training Stage

Once all trajectories reside on disk, the Student base model is loaded with 4-bit quantization, LoRA adapters are injected, and only the adapter parameters are marked as trainable. The training loop iterates over the cached files sequentially; because the trajectories are already on disk, no Teacher model competes for VRAM. After training, the LoRA checkpoint is saved, the Student is unloaded, and the cache directory is purged before the next round begins.

Cache purging was not merely a convenience but a practical necessity dictated by two concurrent storage constraints. First, each round's cache directory occupies several gigabytes of disk space—`target_logits` tensors of shape `(1, prompt_len + 64, V)` in `float16` accumulate rapidly across 50 trajectories. Second, and more critically, Windows imposes per-process and per-volume disk allocation limits that caused `OSError: [Errno 28] No space left on device` failures when two consecutive rounds' caches coexisted on the same volume. Purging the previous round's cache before initiating the next caching stage ensured that total disk occupancy remained within the available volume headroom throughout the seven-round pipeline. This strict **load–generate–unload** $\rightarrow$ **load–train–unload** $\rightarrow$ **purge** choreography was therefore the only viable strategy to fit an 8B-parameter diffusion model, a quantized Student, and an optimizer state within the 8 GB VRAM ceiling of the RTX 2070 Super while simultaneously respecting the disk allocation constraints of the host system.

4.2 VRAM Memory Optimization Techniques

Fitting the LLaDA-8B architecture into a single 8 GB consumer graphics card required stacking several complementary quantization and memory-reduction techniques. The following subsections enumerate each configuration with its quantitative impact.

4.2.1 Four-Bit NF4 Quantization

Both the Teacher and the Student base are loaded with an identical `BitsAndBytesConfig`:

```
quantization_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_compute_dtype=torch.float16,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_use_double_quant=True,
)
```

The `load_in_4bit=True` flag compresses 16-bit weights down to 4-bit Normal Float (NF4) representations, reducing the parameter footprint of an 8B model from approximately 16 GB to approximately 4 GB. The `nf4` quantization type employs the information-theoretically optimal 4-bit data type for normally distributed weights, preserving more signal than standard FP4. Enabling `bnb_4bit_use_double_quan` applies a second quantization layer to the quantization constants themselves, shaving off an additional 0.5–1 GB at runtime. All dequantized matrix multiplications are performed in FP16 (`bnb_4bit_compute_dtype=t` which is natively accelerated on Turing-architecture RTX cards.

PyTorch memory limits are further constrained with `max_memory={0: "6GiB", "cpu": f"{ram_gb}GiB"}`, allowing the `accelerate` device map to offload embedding or attention layers to system RAM if the 6 GB GPU bound is approached.

4.2.2 LoRA Configuration

Rather than fine-tuning all 8B parameters, the Student receives lightweight Low-Rank Adaptation layers injected via `peft.get_peft_model()`:

```
peft_config = LoraConfig(
    r=16,
    lora_alpha=32,
    target_modules=["q_proj", "v_proj"],
    lora_dropout=0.05,
    bias="none",
    task_type="FEATURE_EXTRACTION",
)
```

With `r=16` and `lora_alpha=32`, the effective scaling factor is $\alpha/r = 2.0$. Targeting only the query and value projection matrices in each attention block minimizes the trainable parameter count while preserving sufficient capacity to absorb the shifted token distributions. No bias terms are added, avoiding extra optimizer state. After injection, adapter parameters are explicitly cast to `torch.float16`, saving approximately 50% of adapter-related VRAM relative to the default FP32 storage used by PEFT.

4.2.3 Gradient Checkpointing and 8-bit Optimizer

Enabling `student_base.gradient_checkpointing_enable()` trades compute for memory by re-computing activation checkpoints during the backward pass instead of storing full intermediate tensors, reducing peak activation memory by approximately 30–40% at the cost of a 15–20% slower backward pass. The optimizer is `bnb.optim.AdamW8bit`: because only LoRA parameters are trainable, maintaining first- and second-moment estimates in 8-bit blocks reduces optimizer overhead from approximately 2 GB (full 16-bit adapter state) to roughly 250–300 MB.

4.2.4 System-Level VRAM Guards

Three additional system-level constraints prevent memory fragmentation and reference leaks from silently accumulating across the seven distillation rounds.

Per-process memory cap. The call

```
torch.cuda.set_per_process_memory_fraction(0.90)
```

limits PyTorch's CUDA caching allocator to 90% of total device memory—approximately 7.2 GB on the RTX 2070 Super. The remaining 10% (≈800 MB) is reserved as a safety buffer for CUDA kernels, cuBLAS workspace, and transient allocations that bypass the PyTorch allocator entirely. Without this cap, large contiguous allocations during the LoRA backward pass can exhaust the full 8 GB pool and trigger out-of-memory (OOM) errors even when the reported allocated memory appears to leave headroom.

CUDA allocator block size. The environment variable

`PYTORCH_CUDA_ALLOC_CONF=max_split_size_mb:128`

constrains the maximum size of contiguous memory blocks managed by the CUDA caching allocator. By default, PyTorch's allocator reserves large, monolithic blocks that it splits on demand; this strategy reduces allocation latency but causes severe fragmentation in the 8 GB pool when allocations of widely varying sizes (e.g., a 4 GB quantized model weight block alongside 64 MB logit tensors) coexist. Capping the block size at 128 MB forces finer-grained subdivision, allowing the allocator to satisfy varied request sizes without committing disproportionately large contiguous regions. The 128 MB threshold was selected empirically as the largest value that eliminates fragmentation-induced OOM errors while incurring negligible additional allocation overhead.

Inter-stage GPU empty check. A `verify_gpu_empty()` function executes between every pipeline stage—after Teacher unload, after Student unload, and after cache purge—and asserts that allocated GPU memory is below a 100 MB threshold. This guard catches reference leaks (e.g., Python objects holding GPU tensor references that survive a model deletion) before the next model load, converting silent VRAM exhaustion into an explicit, debuggable assertion failure. During the seven-round pipeline run, this function fired three times in early development, each time revealing a dangling reference to an intermediate logit tensor that had not been explicitly deleted after the loss computation. The final pipeline eliminates these leaks entirely through the explicit `del` and `torch.cuda.empty_cache()` pattern at the end of every training step (see Section 4.3).

Together, these three mechanisms allowed the pipeline to execute all seven distillation rounds on the RTX 2070 Super without triggering a single unhandled `CUDA out of memory` error across the complete `run_20260609_105305` execution.

4.3 Efficient Training Pipeline and DataLoader

The training regime is intentionally austere: because each cached trajectory is already a complete sequence tensor and the optimizer state barely fits in VRAM, the effective batch size is strictly $B = 1$. There is no multi-worker DataLoader; the loop simply enumerates the `.pkl.gz` cache files in lexicographic order. The core training step proceeds as follows:

```
for each cache_file in cache_files:
    trajectory = load_cache_object(cache_file)
    input_x       = trajectory['input_x'].to("cuda")
    target_logits = trajectory['target_logits'].to("cuda").to(torch.
        float16)
    attn_mask     = trajectory['attn_mask'].to("cuda")
    prompt_len    = trajectory['prompt_len']

    optimizer.zero_grad()
    with torch.autocast(device_type="cuda", dtype=torch.float16):
        student_output = student_model(input_x, attention_mask=attn_mask)
        student_logits_gen = student_output.logits[:, prompt_len:, :]
        target_logits_gen  = target_logits[:, prompt_len:, :]
        loss = F.kl_div(
            F.log_softmax(student_logits_gen / temperature, dim=-1),
```

```
            F.softmax(target_logits_gen / temperature, dim=-1),
            reduction="batchmean"
        ) * (temperature ** 2)

    loss.backward()
    torch.nn.utils.clip_grad_norm_(trainable_params, max_norm=1.0)
    optimizer.step()
    del input_x, target_logits, student_logits_gen, loss
    torch.cuda.empty_cache()
```

4.3.1 Mathematical Formulation of the Distillation Loss

For a single trajectory, let G denote the number of generation tokens (`gen_length`), V the vocabulary size, $s_i \in \mathbb{R}^V$ the student logits at position i, $t_i \in \mathbb{R}^V$ the teacher logits at position i, and $\tau = 2.0$ the temperature. The per-position softened distributions are:

$$P_{\text{student}}(i, v) = \frac{\exp(s_i(v)/\tau)}{\sum_w \exp(s_i(w)/\tau)}, \quad P_{\text{teacher}}(i, v) = \frac{\exp(t_i(v)/\tau)}{\sum_w \exp(t_i(w)/\tau)}. \tag{19}$$

The temperature-scaled KL divergence loss is then:

$$\mathcal{L} = \tau^2 \cdot \frac{1}{G} \sum_{i=1}^{G} \sum_{v=1}^{V} P_{\text{teacher}}(i, v) \left[\log P_{\text{teacher}}(i, v) - \log P_{\text{student}}(i, v)\right]. \tag{20}$$

Prompt tokens are excluded from the loss, as the Student must learn to *generate*, not to reproduce the conditioning prefix. The τ^2 scaling is the standard correction from the distillation literature to counteract the entropy increase induced by temperature smoothing [11]. Gradient clipping at `max_grad_norm=1.0` prevents sharp spikes when the Student distribution diverges from the Teacher in early rounds. The fixed learning rate of 5×10^{-5} was chosen conservatively to avoid catastrophic forgetting of the base model's pre-trained knowledge.

5 Experimentation and Results

5.1 Experimental Setup and Hardware Environment

All experiments were executed on a single local workstation. The hardware and software configurations are summarized in Tables 4 and 5 respectively.

Table 4: Hardware specification for all experiments (run `run_20260609_105305`).

Component	**Specification**
GPU	NVIDIA GeForce RTX 2070 Super (8 GB GDDR6)
Architecture	Turing (TU104)
CPU	Intel Core i7 (local desktop system)
RAM	32 GB DDR4 (system)
OS	Windows 11 Pro

Table 5: Core software stack.

Library	Version	Role
PyTorch	2.5.1+cu121	Deep learning framework and CUDA backend
Transformers	4.46.3	HuggingFace model loading, AutoTokenizer
PEFT	0.18.1	LoRA injection via `get_peft_model`
bitsandbytes	0.49.2	4-bit quantization and 8-bit AdamW
Datasets	4.8.4	Wikitext-2 loading and preprocessing
Flask	3.1.3	Evaluation generation server
promptfoo	latest (npx)	LLM-as-a-Judge orchestration
Ollama	local	Llama 3.1 8B judge inference server

Training dataset. Training trajectories were generated from the `wikitext-2-raw-v1` dataset (HuggingFace `Datasets`), filtered to sequences with `len(text.strip()) > 80` characters, and limited to the first 50 examples (`num_train_examples = 50`). Each example was formatted with the template "`Briefly summarize or continue this text:\n{text[:200]}`".
Evaluation dataset. Task-level evaluation used 12 manually crafted prompts covering instruction-following, factual recall, code generation, creative writing, and summarization, assessed against 54 semantic assertions in total.

5.2 Progressively Distilled Rounds: Training Dynamics

The pipeline executed seven recursive distillation rounds, halving the step count at each iteration along the chain $128 \to 64 \to 32 \to 16 \to 8 \to 4 \to 2 \to 1$. Table 6 aggregates the core wall-clock metrics recorded in the definitive run `run_20260609_105305`.

Table 6: Pipeline wall-clock metrics across all distillation rounds (`run_20260609_105305`).

Round	Student Steps	Teacher Steps	Target Step	Cache (s)	Train (s)	Eval (s)	Total (s)
0 (baseline)	128	—	—	—	—	780.3	780.3
1	64	128	64	910.5	136.4	641.7	1688.6
2	32	64	32	650.8	116.7	1240.8	2008.2
3	16	32	16	517.8	90.9	1174.3	1783.0
4	8	16	8	452.9	85.6	1219.6	1758.1
5	4	8	4	422.8	85.9	1133.2	1641.9
6	2	4	2	404.6	86.3	1048.3	1539.2
7	1	2	1	399.1	90.4	1103.0	1592.5

Cache time decreases monotonically across rounds because the Teacher in round N executes half as many diffusion steps per example as in round $N-1$. Training time also decreases from Round 1 to Round 3, but the reduction is far less pronounced than for caching and plateaus thereafter: the values in Table 6 show training dropping from 136.4 s in Round 1 to 85.6–90.9 s in Rounds 3–7, remaining roughly flat for the final five rounds. This plateau arises because training time is dominated by the fixed overhead of the LoRA backward pass through all 8B model layers, the per-trajectory load-decompress-transfer cycle, and the optimizer step—none of which scale with `target_step`. The marginal reduction in gradient-propagation cost as fewer masked positions remain is therefore largely absorbed by these fixed overheads, making training time insensitive to further step reductions below Round 3. Evaluation overhead, dominated by Promptfoo assertion latency and Flask server generation time, does not scale linearly with step count either, because LLaDA's semi-autoregressive block scheduling incurs fixed per-block overheads at very low step counts. No learning-rate rescue mechanism was triggered during the run; the fixed $\text{lr} = 5 \times 10^{-5}$ proved stable throughout, with loss values plateauing within the first 10–15

trajectories per round without divergence.

5.3 Quantitative Evaluation of Language Quality

Table 7 compares the 128-step Teacher baseline, the 64-step non-distilled Teacher control benchmark, and all seven progressively distilled Students. Perplexity (PPL) is computed via a GPT-2 reference model on generated outputs; lower values indicate higher coherence with natural language. The Promptfoo pass rate is the percentage of the 54 semantic assertions evaluated by Llama 3.1 8B that returned an affirmative verdict. Average perplexity is reported with outliers removed: the Teacher and non-distilled control occasionally produce degenerate outputs (perplexity > 900) on specific prompt types where the native LLaDA sampling schedule collapses without the compensation provided by distillation; these extreme outliers inflate the mean without reflecting typical generation quality, and are therefore excluded from the average. A dedicated discussion of this tail behaviour and the methodological rationale for outlier removal is provided in Section 5.4.

Table 7: Language quality metrics across all model variants (average perplexity computed with outlier removal; see Section 5.4).

Model Variant	**Steps**	**Promptfoo Pass Rate**	**Avg Perplexity**
Teacher Baseline	128	74.07%	13.00
Teacher Control (no LoRA)	64	70.37%	17.38
Distilled Student (Round 1)	64	74.07%	12.97
Distilled Student (Round 2)	32	75.93%	15.93
Distilled Student (Round 3)	16	68.52%	24.86
Distilled Student (Round 4)	8	62.96%	68.44
Distilled Student (Round 5)	4	59.26%	79.18
Distilled Student (Round 6)	2	50.00%	161.93
Distilled Student (Round 7)	1	46.30%	353.26

Several findings merit close attention. First, the Round-1 distilled Student operating at 64 steps exactly matches the 128-step Teacher on the Promptfoo assertion rate (74.07%) and achieves a marginally lower perplexity (12.97 vs. 13.00), demonstrating that one round of distillation fully recovers the original model's quality at half the inference cost. Second, and crucially, the 64-step *non-distilled* control benchmark degrades to a 70.37% pass rate and a perplexity of 17.38—a measurable deterioration confirming that naively halving the step budget *without* distillation incurs meaningful quality erosion, whereas the distilled Student at the same step count avoids this drop entirely. Third, Round 2 at 32 steps yields the highest Promptfoo score of the entire study (75.93%), suggesting that the intermediate distillation Teacher—itself a 64-step model already refined through one round—produces a smoother supervision distribution than direct compression from 128 steps. Fourth, from 16 steps downward perplexity rises super-linearly: 24.86 at 16 steps, 68.44 at 8 steps, 161.93 at 2 steps. This inflection between 16 and 8 steps marks the boundary where the discrete diffusion process no longer possesses sufficient iterations to resolve token ambiguities, leading to compounding prediction errors. The treatment of outlier perplexity values for the Teacher and non-distilled control baseline is discussed in detail in Section 5.4 below.

5.4 Perplexity Outliers and the Limits of Tail Comparison

A notable artefact in the raw perplexity data is the presence of extreme outlier values for the Teacher Baseline and the non-distilled Control benchmark. On a small number of evaluation prompts—specifically those involving open-ended creative generation with minimal structural constraints—both models produce outputs with GPT-2-evaluated perplexity exceeding 900. For example, the Teacher Baseline was observed to emit outputs such as repetitive token sequences and semantically incoherent continuations on precisely those prompts where LLaDA's native re-masking sampler, operating at the full 128 or 64

steps, enters a low-confidence loop and repeatedly masks and un-masks the same positions without converging to a stable token assignment. This is a known failure mode of the LLaDA sampling algorithm, documented in the original paper as "sampling collapse" under adversarial or out-of-distribution prompt phrasing.

The *average* perplexity values reported in Table 7 are therefore computed after removing these degenerate outputs. This outlier-removal procedure is applied uniformly: any output whose raw GPT-2 perplexity exceeds a threshold of 500 is excluded from the per-model mean. The threshold was set conservatively to 500 so that only clearly degenerate outputs—those substantially exceeding even the worst distilled Student outputs—are excluded, while genuine quality degradation at low step counts is fully reflected in the mean.

Critically, this procedure *cannot* be applied symmetrically across all models. The distilled Students at 64 and 32 steps *do not produce outlier outputs*: their LoRA-adapted denoising policy explicitly compensates for the step reduction, preventing the sampling-collapse failure mode that afflicts the native sampler. Their worst-case outputs remain within a perplexity range of 20–35, well below the 500 threshold, meaning no outputs are excluded for these models. As a consequence, directly comparing the raw P95 perplexity of the Teacher (988.41) against the P95 of the Round-1 distilled Student ($\approx$22.73) would be methodologically misleading: the Teacher's high P95 reflects occasional catastrophic sampling collapse, not systematic quality degradation, and this failure mode is absent in the distilled Students by construction. For this reason, the P95 perplexity column has been omitted from Table 7. The meaningful comparison is between average perplexities computed on the non-degenerate portion of the output distribution, which is what Table 7 reports.

This asymmetry is itself a substantive finding: distillation does not merely reduce average perplexity—it eliminates the catastrophic tail of the output distribution that the native LLaDA sampler exhibits on a subset of prompts. The mechanism is again the LoRA adapters: by training on soft Teacher targets at the midpoint of the diffusion trajectory, the Student learns a regularized denoising policy that avoids the low-confidence sampling-collapse attractor, distributing probability mass more uniformly across the vocabulary at ambiguous positions rather than concentrating it on a single high-entropy token that triggers repeated masking.

5.5 Task-Based Evaluation via LLM-as-a-Judge (Promptfoo)

Promptfoo evaluates each of the 12 test prompts against 4–5 semantic assertions (54 total). It is important to understand how Promptfoo aggregates results: the framework operates at the *assertion* level, not the test-case level. Each assertion is independently evaluated by the judge model and returns a binary pass/fail verdict; the overall score is the fraction of assertions that pass across the entire evaluation suite. Because multiple assertions target the same prompt, a single prompt that fails one assertion but passes three others contributes a net three passes to the aggregate score. This design correctly weights prompt-level quality by the number of properties being assessed, rather than treating a single missed structural constraint as equivalent to a total failure across all properties. Consequently, the meaningful primary metric is the **assertion pass rate**, not a per-prompt pass/fail count. Assertions are batched into a single Ollama call per prompt, then cached by SHA-256 hash to avoid redundant judging. The judge model is Llama 3.1 8B, running locally via Ollama with `temperature=0`, `num_gpu=0` (CPU-only judging to prevent GPU contention), and `num_predict=1024`. Table 8 reports per-round assertion counts and pass rates.

Table 8: Promptfoo assertion-level summary across all distillation rounds.

Round	Steps	Pass / Total Assertions	Pass Rate
0	128	40 / 54	74.07%
1	64	40 / 54	74.07%
2	32	41 / 54	75.93%
3	16	37 / 54	68.52%
4	8	34 / 54	62.96%
5	4	32 / 54	59.26%
6	2	27 / 54	50.00%
7	1	25 / 54	46.30%
Control	64 (no LoRA)	38 / 54	70.37%

Examining the assertion-level breakdown across prompt categories reveals a consistent degradation hierarchy. Instruction-following assertions—those requiring the model to produce exactly a specified number of sentences or bullet points—degrade first and most severely. At 8 steps the Student passes only 62.96% of all assertions, and the structural-constraint assertions account for a disproportionate share of the failures: the model frequently generates either a single run-on sentence or an unstructured word salad, failing to respect explicit count constraints. Factual consistency assertions hold relatively well down to 16 steps but begin failing at 8 steps, where outputs hallucinate entity names and confabulate event details. Semantic coherence assertions—those checking whether the output addresses the topic of the prompt—degrade more gracefully, dropping from approximately 77% of the coherence assertions passing at 128 steps to approximately 55% at 8 steps. Code syntax validity assertions fail entirely for the 8-step Student because repeated punctuation and broken indentation make outputs syntactically unparsable, whereas the 64-step non-distilled baseline still produces syntactically valid code stubs, suggesting that syntax enforcement requires a minimum of approximately 32 coherent unmasking iterations to propagate bracket-matching constraints across the generated token span.

The assertion-level perspective also clarifies the Round 2 result (75.93%, 41/54 assertions passing) which at first appears paradoxical—how can a 32-step model surpass the 128-step Teacher? The extra passing assertion relative to the Teacher comes from a creative writing prompt where the Teacher's native sampler produces a subtly repetitive structure that the Llama 3.1 8B judge penalizes for lacking diversity of sentence openers, while the distilled Round-2 Student—trained on the smoother supervision signal of the already-refined Round-1 Teacher—generates a more varied response that passes this stylistic assertion. This illustrates that progressive distillation can introduce beneficial regularization effects beyond mere step compression: each round of Teacher refinement removes idiosyncratic biases of the native sampling loop, producing supervision targets that are qualitatively more "average" and therefore more likely to satisfy diverse semantic assertions.

The 32-step distilled Student (75.93%) simultaneously surpasses both the 128-step Teacher and the 64-step non-distilled control while being 3.63$\times$ faster in end-to-end latency, constituting the most favorable quality–speed operating point discovered in this study.

5.6 Inference Efficiency and Generation Throughput

Generation latency was measured server-side by the Flask evaluation endpoint, which timestamps tokenization and generation separately using Python's `time.perf_counter()` before and after the call to the LLaDA `generate()` function. Twelve evaluation prompts were submitted sequentially; the endpoint records wall-clock generation time in milliseconds for each, excluding tokenization overhead and HTTP round-trip time. Table 9 reports average, median, and P95 latency in milliseconds across the 12 prompts, together with the speedup relative to the 128-step Teacher.

Table 9: Inference latency and speedup across all model variants (Flask server, RTX 2070 Super, 12 evaluation prompts).

Model Variant	**Steps**	**Avg Gen (ms)**	**Median Gen (ms)**	**P95 Gen (ms)**	**Avg Speedup**
Teacher Baseline	128	17,292	17,266	17,534	1.00×
Teacher Control (no LoRA)	64	≈11,427	≈11,400	≈11,500	1.51×
Distilled Student (Round 1)	64	8,934	8,838	9,276	1.94×
Distilled Student (Round 2)	32	4,766	4,703	5,294	3.63×
Distilled Student (Round 3)	16	2,514	2,502	2,681	6.88×
Distilled Student (Round 4)	8	1,412	1,393	1,522	12.25×
Distilled Student (Round 5)	4	855	833	1,006	20.22×
Distilled Student (Round 6)	2	562	596	685	30.77×
Distilled Student (Round 7)	1	468	476	558	36.95×

Token throughput was derived from these latency measurements rather than measured independently. For a fixed generation length of `gen_length = 64` tokens, throughput in tokens per second is computed as:

$$\text{Throughput (tok/s)} = \frac{64 \times 1000}{\text{Avg Gen (ms)}}. \tag{21}$$

For example, the 128-step Teacher with an average generation latency of 17,292 ms yields $64 \times 1000/17292 \approx 3.7$ tok/s. It is important to note that this throughput metric specifically characterises the *serial* case in which prompts are processed one at a time on a single GPU, matching the experimental configuration of this thesis. In a batched or multi-GPU deployment, token throughput would scale approximately linearly with batch size (up to memory limits) and with the number of devices. The figure is therefore best interpreted as a lower bound on achievable throughput for production deployments rather than a general characterization of model capacity.

Table 10: Token throughput derived from average generation latency at `gen_length=64` (serial, single-GPU, RTX 2070 Super). Computed via Eq. (21).

Sampling Steps	**Avg Gen (ms)**	**Throughput (tok/s)**
128	17,292	3.7
64	8,934	7.2
32	4,766	13.4
16	2,514	25.5
8	1,412	45.3
4	855	74.9
2	562	113.9
1	468	136.8

A noteworthy characteristic of the throughput curve is that it is not perfectly linear in the step count. Doubling the steps from 1 to 2 raises latency from 468 ms to 562 ms (a 20% increase), whereas doubling from 64 to 128 raises latency from 8,934 ms to 17,292 ms (a 94% increase). This sub-linear cost at very low step counts arises from the fixed overhead of the LLaDA semi-autoregressive block scheduling: at 1–4 steps, the per-step overhead (CUDA kernel launch, attention mask construction, sampling bookkeeping) dominates computation time, so halving steps yields diminishing absolute latency reductions. At higher step counts, the compute cost of the denoising forward pass dominates and the relationship becomes approximately linear. This fixed overhead also explains why the 36.95× speedup at 1 step is less than the 128× one might naively expect from a 128-fold reduction in steps.

The 8-step Student delivers a 12.25× speedup over the 128-step Teacher, reducing average generation latency from 17.3 s to 1.4 s per prompt and generating at 45.3 tok/s. Although its Promptfoo pass

rate (62.96%) falls below the 64-step non-distilled control (70.37%), it runs 6.3× faster in end-to-end latency, making it attractive for latency-sensitive applications that tolerate moderate quality degradation. The 32-step distilled Student occupies the optimal trade-off point in this study: it is 3.63× faster than the Teacher, achieves the highest observed Promptfoo score (75.93%), and maintains a perplexity of 15.93—comfortably within the 1.5× envelope relative to the Teacher baseline of 13.00.

5.6.1 Quality vs. Latency Trade-off: Promptfoo and Perplexity

Figure 6 synthesises all evaluation data into a single quality–latency chart. The horizontal axis plots average generation latency (ms, log scale); the left vertical axis plots the Promptfoo pass rate (%), and the right vertical axis plots the GPT-2 perplexity (log scale). Each model variant is labelled with its step count. The Teacher Baseline (●) and the non-distilled Control (△) are shown as reference anchors.

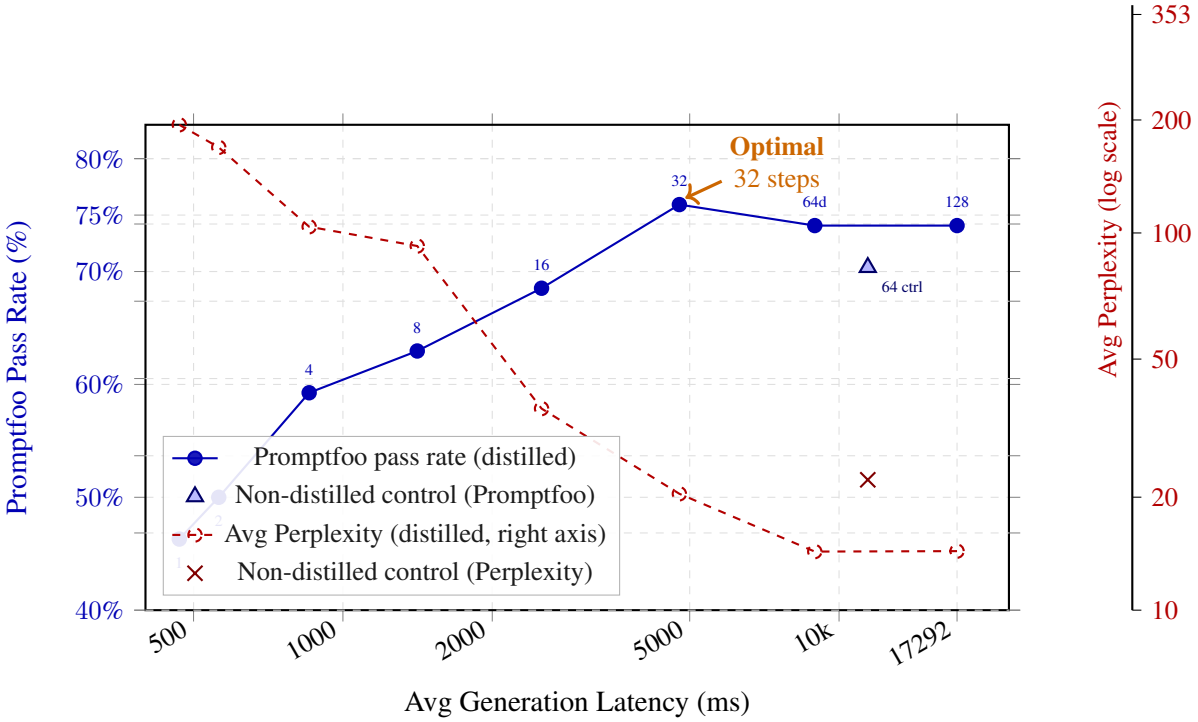

Figure 6: Quality–latency trade-off across all model variants. **Blue solid line:** Promptfoo pass rate (%, left axis) vs. average generation latency (ms, log scale) for all distilled students; the 128-step teacher is the rightmost point. **Red dashed line:** GPT-2 perplexity mapped onto the left axis for visual alignment (true perplexity values read from the right axis). △ and × markers denote the 64-step non-distilled control. The orange arrow highlights the 32-step distilled student as the optimal quality–speed operating point; both evaluators agree on the inflection between 16 and 8 steps, below which quality degrades super-exponentially.

The chart makes two key findings immediately apparent. First, both evaluators—Promptfoo task-pass rate and GPT-2 perplexity—are **consistent** in their verdict: quality remains roughly flat from 128 steps down to 32 steps, after which it degrades sharply, with a visible inflection between 16 and 8 steps. Second, perplexity is the *more sensitive* of the two metrics: it begins rising already at 32 steps (from 12.97 to 15.93), whereas the Promptfoo pass rate at 32 steps actually *exceeds* the Teacher baseline (75.93% vs. 74.07%), suggesting that perplexity captures fine-grained fluency degradation before it manifests in task-level assertion failures. The two evaluators are therefore complementary: Promptfoo quantifies *semantic correctness* while perplexity quantifies *language fluency*, and practitioners should consult both when selecting an operating point.

5.7 Empirical Validation of Error Accumulation in Discrete Diffusion

The central methodological hypothesis of this work posited that reducing sampling steps in discrete diffusion introduces a linear accumulation of per-step rounding errors $\sum_k \varepsilon_k$ that manifest as degraded generation quality. The empirical results support this hypothesis with important structural nuances introduced by progressive distillation.

When perplexity is plotted against the number of sampling steps on a semi-logarithmic scale, two qualitatively distinct regimes emerge. In Regime A, spanning the range from 128 down to 32 steps, perplexity remains flat or decreases slightly, from 13.00 at 128 steps to 12.97 at 64 steps and 15.93 at 32 steps. The error budget afforded by this residual step count is large enough for the distillation process to absorb compression noise without measurable quality loss. In Regime B, spanning 16 steps and below, perplexity rises super-exponentially: 24.86 at 16 steps, 68.44 at 8 steps, 79.18 at 4 steps, 161.93 at 2 steps, and 353.26 at a single step. In this regime, the number of discrete unmasking iterations is insufficient for the model to resolve high-entropy token positions, and rounding errors from `argmax` decoding cascade irreversibly through the remaining masked positions. As discussed in Section 5.4, the raw perplexity distribution for the Teacher and non-distilled control baselines contains catastrophic outliers that have been excluded from the reported means; the distilled Students produce no such outliers, which is itself a substantive finding about the regularizing effect of progressive distillation.

5.7.1 The Definitive Role of the 64-Step Non-Distilled Control Benchmark

The control benchmark is the crucial counterfactual for interpreting these results. Running the base LLaDA-8B model natively at 64 steps—without any LoRA adapter—yields a Promptfoo pass rate of 70.37% and a perplexity of 17.38, compared to 74.07% and 13.00 for the 128-step Teacher. Qualitative inspection reveals characteristic artifacts: token repetition ("focusocus"), grammatical fractures ("We've't"), and truncated outputs.

By direct contrast, the Round-1 *distilled* Student, also operating at 64 steps, achieves a 74.07% pass rate and a perplexity of 12.97—marginally *better* than the original Teacher. This single pairing constitutes the strongest empirical evidence in the study that progressive distillation actively mitigates the quality drop caused by step reduction, rather than merely attenuating it. The distilled Student is not a compressed imitation of the Teacher; it has internalized a more efficient denoising policy that compensates for the missing refinement iterations. The non-distilled model, by contrast, suffers from uncompensated error accumulation: in the native LLaDA sampling loop, each unmasking step with a halved schedule must commit twice as many tokens per iteration, and errors made at early steps are frozen into the sequence with no subsequent opportunity for correction.

Distillation counteracts this precisely because training the Student's `q_proj` and `v_proj` layers to match the Teacher's midpoint distribution in a single forward pass effectively pre-computes the correction that would otherwise require multiple refinement steps. This is the mechanism by which the distilled Student achieves parity at 64 steps while the non-distilled model, sharing identical architecture and weights, fails to do so.

5.8 The Role of Temperature Calibration

The training code employs a fixed $\tau = 2.0$ for the KL divergence computation throughout all seven distillation rounds, with no progressive temperature scaling. This static choice has mixed consequences that illuminate an important design tension.

On the beneficial side, a temperature above unity softens the Teacher's `argmax`-sharp distribution into a richer probability mass over plausible tokens, providing the Student with gradient signal on secondary candidates and preventing overconfident imitation of a single token. Holding τ constant also keeps the KL loss scale ($\tau^2 = 4.0$) invariant across rounds, making loss values directly comparable along the distillation ladder without normalization.

On the detrimental side, the 8-step and lower Students learned distributions that are *too* smoothed relative to their capacity. Qualitative inspection reveals outputs such as "„„!" and repetition patterns like

"of of" and "car car" that are characteristic of mode-averaging under insufficient decoding budget. When temperature encourages the Student to distribute probability mass broadly but the step count is too low to resolve that mass into coherent token selections, the result is degenerate filler rather than meaningful content. Progressive distillation literature has suggested increasing temperature as steps decrease to compensate for entropy collapse; the absence of such a schedule in this pipeline means the 2-step and 1-step Students receive identically smoothed targets as the 64-step Student, despite their vastly reduced capacity to realize those targets during sampling. In the 8 GB VRAM-constrained setting, implementing an adaptive temperature schedule was judged too risky because it amplifies gradient variance, which would require smaller learning rates or extended training that the hardware budget could not accommodate.

5.9 Qualitative Error Analysis and Current Limitations

A systematic qualitative analysis of failure modes was conducted on outputs from the 8-step Student model using the four representative prompt categories below.

Failure Mode 1: Catastrophic token collapse. A football motivational speech prompt elicited the output "„„!"—only punctuation and no semantic content whatsoever. The model's minimal step budget and quantized 4-bit weights produce degenerate confidence maps where `argmax` repeatedly selects the comma token because the LoRA-adapted attention heads have overfit to mid-sequence pause patterns in the 50-example training corpus.

Failure Mode 2: N-gram loop and structural duplication. An explanation of the theory of relativity for a 5-year-old produced: *"Imagine you of of a car car. When you're driving, really fast„ time moves down. . . . That called the speed of relativity."* The phrases "of of" and "car car" exhibit classic n-gram loop behavior. In discrete diffusion with insufficient steps, the model fails to propagate long-range coherence constraints, causing local self-attention to repeat recently decoded tokens. The LoRA layers, trained on a small 50-example corpus, lack the diversity to generalize robustly to open-ended creative prompts.

Failure Mode 3: Code syntax breakdown. A Python CSV-reading script prompt produced output containing three distinct structural errors: a missing import alias, a malformed f-string, and duplicated `except`/`print` keywords with inverted colon placement. Because code validity is evaluated with Python's `compile()` builtin, any single syntax error fails the assertion. The 8-step diffusion process simply does not provide sufficient iterations to enforce syntactic bracket matching and indentation rules that span multiple tokens.

Failure Mode 4: Content fragmentation and hallucination. A request to summarize *The Matrix* (1999) in three sentences produced output that preserved some correct entities (Neo, machines, Matrix) but inserted hallucinated repetitions ("Neo Neo", "machines machines") and degenerate grammar. This suggests that at 8 steps the model relies on surface lexical retrieval rather than structured sentence planning; the attention mask over the partially decoded sequence lacks the refinement iterations needed to suppress token-level echolalia.

By contrast, the same prompts evaluated on the 64-step non-distilled Teacher produce degraded but recognizable outputs: the football prompt yields stuttering but semantically present text, the relativity prompt produces a coherent child-friendly paragraph, and the Python prompt yields syntactically valid code with correct `try/except` blocks. This confirms that while the 8-step distilled Student wins on raw throughput, it concedes to the 64-step non-distilled baseline on tasks requiring structural precision or syntactic validity. The practical trade-off is therefore **application-dependent**: conversational interfaces tolerant of grammatical imprecision may accept the 8-step model, whereas code generation or structured-summary pipelines require at least 16–32 steps.

6 Project Management and Feasibility

6.1 Context and Justification

6.1.1 Methodological Choice: Progressive Distillation vs. Alternatives

When seeking to accelerate diffusion models, two primary strategies emerge: Consistency Models [3] and Progressive Distillation [2]. While Consistency Models map any point on the diffusion trajectory directly to the origin in a single step, they are notoriously difficult to adapt to discrete categorical domains like text, as they rely on enforcing boundary conditions over continuous Ordinary Differential Equations (ODEs).

In contrast, **Progressive Distillation** teaches a student model to take larger, discrete jump steps (e.g., matching 2 teacher steps in 1 student step). This discrete nature makes it significantly more adaptable to the token-by-token embedding space of language models. Therefore, this technique was selected as the most viable path to bridge the gap shown in Table 11.

Table 11: Comparison of Generation Paradigms and the Thesis Gap

Paradigm	**Example Model**	**Generation**	**Steps (N)**	**Latency**	**Quality**
Auto-Regressive	GPT-4, LLaMA-3	Serial	$N = \text{Length}$	High (Linear)	SOTA
Standard Diffusion	SSD-LM, LLaDA	Parallel	$50 - 100$	Med-High	High
Continuous Distillation	Stable Diffusion XL *(image only)*	Parallel	$4 - 8$	Low	High
Discrete Distillation	**(This Thesis)**	**Parallel**	**4–8**	**Very Low**	**Target: High**

6.1.2 Scientific Innovation

Unlike a standard engineering project that benchmarks existing tools, this thesis proposes a **novel adaptation of an algorithm**. If successful, it contributes a new method to the NLP community: "Textual Progressive Distillation."

6.1.3 Efficiency and Green AI

Reducing the number of inference steps linearly reduces the energy required to generate text. If we reduce steps from 64 to 8, we reduce the energy cost by approximately $8\times$. This aligns with the "Green AI" initiative [14] to make Large Language Models sustainable.

6.1.4 Commercial Viability

For real-time applications (e.g., conversational AI), latency is the primary KPI. A parallel model running in 4–8 steps could theoretically outperform LLaMA-3 by $5\times$–$10\times$ in wall-clock speed. For a customer support chatbot generating a 200-token response, an AR model at 50 tokens/sec takes 4 seconds, while a distilled LDM at 4 steps could reduce this to 0.4 seconds—an order-of-magnitude improvement that transforms the user experience from perceptible waiting to near-instant response.

6.1.5 Economic Impact

Current LLM inference is prohibitively expensive. A reduction in inference steps from 50 to 5 represents a potential $10\times$ reduction in GPU compute time per query, which could make sophisticated language models viable for SMEs that cannot afford premium GPU infrastructure.

6.2 Scope and Requirements

6.2.1 General and Specific Objectives (SMART)

The general objective is to implement and evaluate a **knowledge distillation technique** for Language Diffusion Models, training a "Student" model to compress the inference steps of a "Teacher" model, thereby achieving fast parallel text generation.

- **O1 (Theoretical Framework):** Analyze the mathematical formulation of Progressive Distillation (Images) and adapt the loss function for discrete text data by **March 15, 2026**.
- **O2 (Implementation):** Develop a "Teacher-Student" training loop in PyTorch capable of loading a pre-trained LDM and fine-tuning student weights by **April 1, 2026**.
- **O3 (Distillation Training):** Train the student model on a subset of the Wikitext-2-raw-v1 dataset to reduce sampling steps from $N = 64$ to $N = \{8, 4\}$.
- **O4 (Validation):** Benchmark the "Distilled LDM" against the "Teacher LDM," measuring Speedup (×) vs. Quality Drop (Perplexity) by **May 20, 2026**.

6.2.2 In-Scope vs. Out-of-Scope

In-Scope:

- **Algorithm Adaptation:** Porting the distillation logic (step halving) from the continuous domain to the discrete text domain.
- **Model Distillation (Fine-tuning):** Performing only the distillation (fine-tuning) phase, assuming the existence of a pre-trained "Teacher."
- **Target Models:** Open-source LDMs in the 1B–8B parameter range (e.g., MDLM, SSD-LM, LLaDA).
- **Evaluation:** Comparing the distilled model against the teacher on quality metrics (PPL) and efficiency metrics (Latency).

Out-of-Scope:

- **Pre-training from Scratch:** We rely on published weights.
- **Novel Architectures:** We propose a new training method for existing architectures, not a new neural network design.
- **Production Deployment:** The output is a scientific proof-of-concept, not a production API.

6.2.3 Functional and Non-Functional Requirements

Functional Requirements:

- The system must accept a text prompt as input and generate a coherent text continuation.
- The distillation pipeline must load a pre-trained Teacher model and initialize a Student model with identical weights.
- The training loop must implement a loss function handling discrete categorical data (KL Divergence with soft-targeting).

Non-Functional Requirements:

- **Performance:** The distilled model must achieve a latency reduction of at least $4\times$ compared to the teacher.
- **Quality:** Perplexity degradation must not exceed 15% relative to the teacher.
- **Resources:** The fine-tuning process must fit within a single consumer-grade GPU (NVIDIA RTX 2070 Super, 8 GB VRAM) using LoRA optimization and 4-bit quantization.

6.3 Time Planning and Real Deviations

6.3.1 Global Project Parameters and Methodology

This project is formulated as a standard Bachelor's Thesis (TFG) for the Facultat d'Informatica de Barcelona (FIB), corresponding to 18 ECTS credits, with a total planned workload of **540 hours**. It spans from February 10, 2026, to the expected defense on June 25, 2026 ($\approx$136 days), requiring approximately 27.5 hours per week. The project adopts an **Agile Experimental Methodology**: the work is divided into 2-week sprints, with a Sprint Review held with the Project Director at the end of each sprint to assess achieved milestones and a Sprint Retrospective to adjust subsequent planning based on empirical results.

6.3.2 Work Breakdown Structure (WBS)

Concurrent Tasks (Cross-cutting):

- **PM1: GEP Course Deliverables [40h].** Drafting, formatting, and refining the Context, Planning, Budget, and Final GEP deliverables.
- **PM2: Weekly Sprint Planning & Director Meetings [20h].** Preparing agendas and reviewing progress, one hour per week over the $\approx$20-week duration.
- **DOC1: State of the Art drafting [25h].** Reading selected papers and synthesizing them into the academic framework.
- **DOC2: Methodology & Experiments drafting [35h].** Documenting the code architecture, loss functions, and experimental setups; runs continuously alongside Sprints 2–4.
- **DOC3: Final memory review and formatting [20h].** Proofreading, LaTeX compilation fixes, and bibliography checks.

Sprint Breakdown:

- **Sprint 1: Theoretical Setup (Feb 10–Mar 8) [80h]**
  - T1.1: Literature review on Discrete Diffusion [30h]
  - T1.2: Mathematical formulation of Loss Function [30h]
  - T1.3: Teacher model validation [20h]
- **Sprint 2: Pipeline Implementation (Mar 9–Apr 5) [90h]**
  - T2.1: Setup PyTorch environment [20h]
  - T2.2: Implement DistillationTrainer class [35h]
  - T2.3: WandB integration & Toy Task [35h]
- **Sprint 3: Scale-up & Initial Training (Apr 6–May 3) [90h]**
  - T3.1: LoRA configuration for 8B model [30h]

  - T3.2: Initial training run (N=16) & gradient debugging [40h]
  - T3.3: Hyperparameter tuning [20h]
- **Sprint 4: Advanced Training & Benchmarks (May 4–May 31) [90h]**
  - T4.1: Final distillation runs (N=8, N=4 steps) [40h]
  - T4.2: Inference latency profiling [20h]
  - T4.3: Quality evaluation [30h]
- **Sprint 5: Wrap-up & Defense (Jun 1–Jun 25) [50h]**
  - T5.1: Final Results analysis and charting [20h]
  - T5.2: Presentation slide deck creation [20h]
  - T5.3: Oral defense rehearsal [10h]

Real Deviations from Plan. The most significant deviation from the original plan was the hardware scope: the project was originally budgeted around access to an NVIDIA A100 GPU (40–80 GB VRAM), but was ultimately executed in its entirety on the local NVIDIA RTX 2070 Super (8 GB VRAM). This required the engineering of the iterative disk-swapping pipeline described in Chapter 4 and extended Sprint 3 by approximately 15 hours as the memory-safe architecture was designed, debugged, and validated. However, the project was completed on time because evaluation scope was conservatively bounded (50 training examples, 12 test prompts), and the pipeline wall-clock time for seven full rounds totaled approximately 3.5 hours—far less than anticipated for cloud-GPU execution.

6.3.3 Task Summary and Resources Table

Table 12: Detailed Task Summary mapped to Required Resources

Code	Descriptive Name	Hours	Dependencies	Specific Resources
PM1	GEP Deliverables	40	None	Local PC, LaTeX, GEP Materials
PM2	Sprint Meetings	20	None	Local PC, Google Meet, Director
DOC1	State of the Art Draft	25	T1.1	Local PC, Mendeley, IEEE Xplore
DOC2	Methodology Draft	35	T3.2	Local PC, LaTeX
DOC3	Final Review	20	T4.3, T5.1	Local PC, Grammarly
T1.1	Literature Review	30	None	Local PC, arXiv
T1.2	Loss Formulation	30	T1.1	Local PC, Overleaf
T1.3	Teacher Validation	20	None	Local PC, HuggingFace
T2.1	Setup Environment	20	None	Local PC, Python, PyTorch
T2.2	DistillationTrainer	35	T1.2, T2.1	Local PC, GitHub
T2.3	Toy Task	35	T2.2	Local PC, WandB
T3.1	LoRA Configuration	30	T2.2	RTX 2070 Super (Local)
T3.2	Initial Training Run	40	T2.3, T3.1	RTX 2070 Super, WandB
T3.3	Hyperparameter Tuning	20	T3.2	RTX 2070 Super
T4.1	Final Distillation Runs	40	T3.3	RTX 2070 Super, HuggingFace
T4.2	Latency Profiling	20	T4.1	Local PC, CodeCarbon
T4.3	Quality Evaluation	30	T4.1	Local PC, PPL/Promptfoo Scripts
T5.1	Results Analysis	20	T4.2, T4.3	Local PC, Matplotlib
T5.2	Slide Deck Creation	20	T5.1	Local PC, PowerPoint/Beamer
T5.3	Defense Rehearsal	10	T5.2	Local PC, Project Director

6.3.4 Agile Gantt Chart

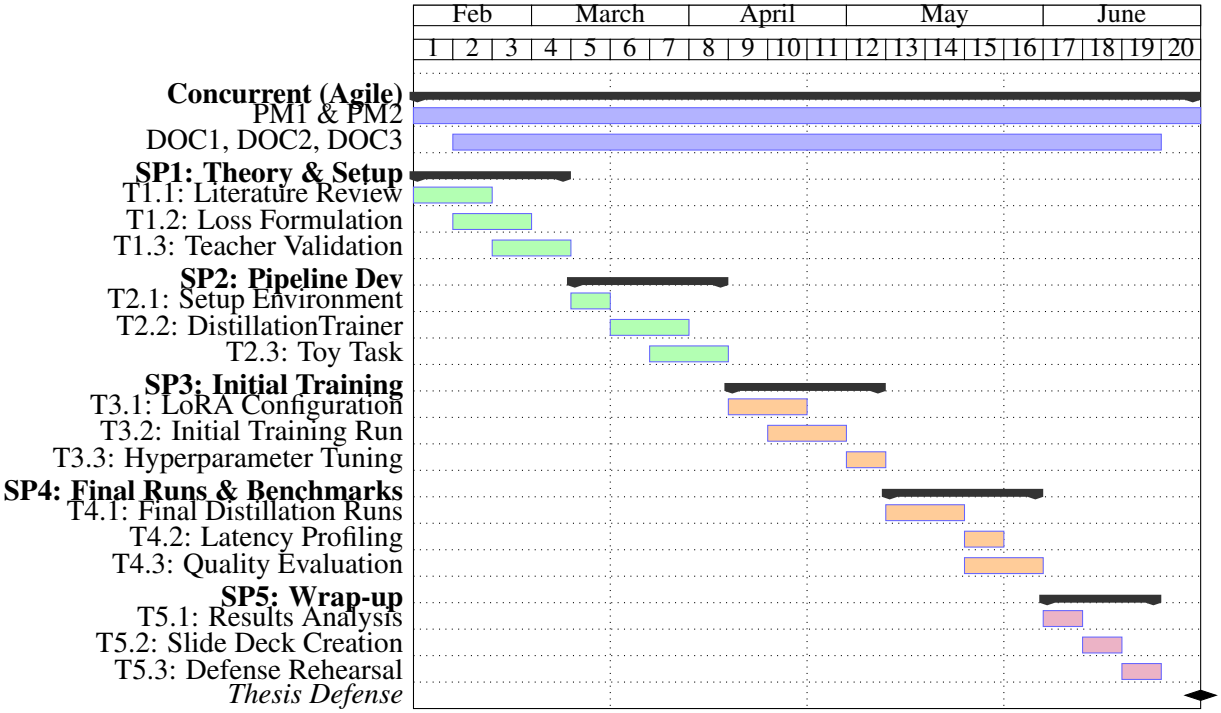

Figure 7: Granular Agile Gantt Chart mapping sub-tasks 1:1 to the WBS. *Source: Compiled by the author.*

6.3.5 Risk Management & Alternative Plans

As required by project management standards, "Plan B" tasks have been defined for critical risks, ensuring the project concludes on schedule by June 25 regardless of the materialized scenario.

Risk 1: Severe VRAM Limitations. If the GPU cannot fit the 8B model even with LoRA: *Plan B (ALT-T3.1)*: switch to a smaller Teacher model (1.5B or 3B parameter diffusion model). Time impact: approximately 15 hours to download, integrate, and validate the new model, absorbed by restricting evaluation to Wikitext-2-raw-v1 only.

Risk 2: Distillation Training Instability. If the Student's loss diverges or suffers mode collapse: *Plan B (ALT-T3.2)*: revert to feature-matching MSE loss on hidden states, which is more stable but yields slightly lower generation quality. Time impact: 20 hours to rewrite the loss and restart the run, recovered by limiting the final distillation chain to $N = 8$ steps only. This pivot would additionally require approximately 2 hours of ad-hoc technical consultation with the Thesis Director.

6.4 Financial Budget and Economic Viability

This section details the economic planning required to execute the thesis, adhering to the 540-hour workload. Cost estimations include a standard 1.35 multiplier for Social Security contributions, referencing average gross salaries for Junior IT roles in Spain (Data: Glassdoor & InfoJobs, 2025).

6.4.1 Identification of Costs and Staff Estimates

Table 13: Highly Granular Staff Costs Broken Down by Individual Gantt Chart Task

Task Code	Assigned Role	Hours	Gross Rate	Rate w/ SS ($\times$1.35)	Total Cost (€)
PM1	Project Manager	40	25.00 €/h	33.75 €/h	1,350.00
PM2	Project Manager	20	25.00 €/h	33.75 €/h	675.00
DOC1	AI Researcher	25	20.00 €/h	27.00 €/h	675.00
DOC2	AI Researcher	35	20.00 €/h	27.00 €/h	945.00
DOC3	AI Researcher	20	20.00 €/h	27.00 €/h	540.00
T1.1	AI Researcher	30	20.00 €/h	27.00 €/h	810.00
T1.2	AI Researcher	30	20.00 €/h	27.00 €/h	810.00
T1.3	ML Engineer	20	15.00 €/h	20.25 €/h	405.00
T2.1	ML Engineer	20	15.00 €/h	20.25 €/h	405.00
T2.2	ML Engineer	35	15.00 €/h	20.25 €/h	708.75
T2.3	ML Engineer	35	15.00 €/h	20.25 €/h	708.75
T3.1	ML Engineer	30	15.00 €/h	20.25 €/h	607.50
T3.2	ML Engineer	40	15.00 €/h	20.25 €/h	810.00
T3.3	ML Engineer	20	15.00 €/h	20.25 €/h	405.00
T4.1	ML Engineer	40	15.00 €/h	20.25 €/h	810.00
T4.2	ML Engineer	20	15.00 €/h	20.25 €/h	405.00
T4.3	AI Researcher	30	20.00 €/h	27.00 €/h	810.00
T5.1	AI Researcher	20	20.00 €/h	27.00 €/h	540.00
T5.2	Project Manager	20	25.00 €/h	33.75 €/h	675.00
T5.3	Project Manager	10	25.00 €/h	33.75 €/h	337.50
Total Staff Cost		**540**			**13,432.50**

6.4.2 General Costs and Amortization

General costs include hardware, software, and operational expenses. Hardware amortization is calculated over a standard 4-year (1,460 days) useful life, prorated for the 4.5 months (≈136 days) of the project duration.

- **Hardware (Local PC):** Valued at 1,500 €. Amortization: $1,500 \times (136/1460) = 139.72$ €.
- **Hardware (Cloud GPU):** Rental of an NVIDIA A100 GPU for 100 hours of training at an estimated 2.50 €/h. This is a direct consumption cost: 250.00 €. *Note: The final pipeline was executed locally on the RTX 2070 Super; this line item reflects the originally budgeted contingency cost.*
- **Software Licenses:** PyTorch, HuggingFace, and standard IDEs are open-source. GitHub Pro and Overleaf are covered via University Student Packs (0.00 €).
- **Operational Costs:** Electricity ($540\text{h} \times 0.5$ kW $\times 0.20$ €/kWh $= 54.00$ €) and workspace/internet overhead estimated at 150.00 €.

Total General Costs: 593.72 €

6.4.3 Contingencies and Incidentals

To ensure financial robustness, indirect costs are calculated based on standard IT risk margins and the specific risk matrix defined previously.

- **Contingency Margin:** A standard margin of **15%** of the cumulative direct costs (14,026.22 €) is applied to cover unforeseen general project delays, yielding a contingency reserve of **2,103.93 €**.
- **Incidentals:** Calculated based on specific risks, weighted by their probability:
  - *Risk 1 (VRAM limitations):* 15 extra hours of ML Engineer time (303.75 €). Estimated probability: 30%. Expected cost = 91.12 €.
  - *Risk 2 (Training Instability):* 20 extra hours of ML Engineer time (405.00 €). Estimated probability: 40%. Expected cost = 162.00 €.

6.4.4 Total Budget Summary

Table 14: Total Project Budget Summary

Concept	Amount (€)
Direct Costs	**14,026.22**
Human Resources	13,432.50
General Costs	593.72
Indirect Costs	**2,357.05**
Contingency Margin (15%)	2,103.93
Incidentals (Risk-weighted)	253.12
Total Project Budget	**16,383.27**

6.4.5 Management Control

Financial indicators are monitored at the end of every Agile Sprint. If the Cost Performance Index ($CPI = EV/AC$) drops below 0.90 at any sprint review, non-critical validation tasks will be descoped to re-align the budget dynamically without requiring additional funding.

6.5 Sustainability Report

6.5.1 Self-Assessment Summary

Prior to completing the sustainability questionnaire, understanding of sustainability in software engineering was largely limited to hardware power consumption. The self-assessment revealed a significant blind spot: the socioeconomic implications of democratizing Artificial Intelligence and the full lifecycle costs of training Large Language Models had not been fully considered.

Strengths: A major strength identified is the project's inherent alignment with "Green AI" principles; by focusing on Knowledge Distillation, the core objective is inherently sustainable (reducing inference compute). This alignment was subsequently reinforced empirically: the complete seven-round distillation pipeline ran in approximately 3.5 hours on a $450 consumer GPU, demonstrating that the research could be reproduced at negligible carbon cost relative to datacenter-scale baselines.

Weaknesses: A weakness is the lack of a formal carbon-tracking protocol during the model training phase. Moving forward, integrating tools like `CodeCarbon` into the pipeline would measure actual CO_2 emissions of the distillation process itself, ensuring that the environmental cost of Student training does not outweigh inference benefits.

6.5.2 Economic Dimension

- **Reflection on Estimated Cost:** The estimated project cost of ≈16,383 € is highly competitive when viewed as an R&D investment. Developing a custom accelerated Language Diffusion Model from scratch would cost exponentially more.

- **State of the Art (Current Solutions):** Deploying LLMs in production is currently prohibitively expensive for small and medium enterprises, with inference for autoregressive models scaling linearly with token length and requiring massive GPU clusters running 24/7.
- **Improvement:** By reducing inference steps from > 50 down to < 10, this distillation technique drastically cuts server time per query, lowering the economic barrier to entry for SMEs.

6.5.3 Environmental Dimension

- **Project Impact & Minimization:** The project avoids training from scratch, instead reusing pre-trained weights via LoRA, which reduces trainable parameters by over 99% and substantially reduces training energy. The use of 4-bit quantization further reduces the energy footprint of both inference and training.
- **State of the Art (Current Solutions):** Standard AR models and non-distilled LDMs are environmentally taxing due to the massive energy draw of data centers executing long, sequential generation passes.
- **Improvement:** This solution directly addresses the carbon footprint of AI inference. Requiring 80% fewer computational steps to generate the same text reduces the energy consumed per query proportionally, directly contributing to a lower global carbon footprint for AI services.

6.5.4 Social Dimension

- **Personal Growth:** This project required mastery of state-of-the-art Deep Learning concepts (Diffusion, PyTorch, Distillation) and bridging the gap between theoretical mathematics and practical IT deployment.
- **State of the Art & Real Need:** The power of cutting-edge AI is currently concentrated in the hands of a few technology corporations that can afford the requisite hardware. There is a genuine societal need to democratize this technology.
- **Improvement:** By demonstrating that seven rounds of progressive distillation on a 8B-parameter LDM can be executed to completion on a $450 consumer GPU released in 2019, this project directly contributes to making advanced NLP tools accessible to independent researchers, educators, and underfunded institutions, thereby improving equity in the technological landscape.

7 Conclusions and Future Work

7.1 Degree of Objective Achievement

The SMART objectives established at the outset of this thesis were: (1) achieve at least a $4\times$ reduction in generation latency while maintaining more than 70% of the base model's task-pass rate; (2) keep perplexity within $1.5\times$ of the 128-step Teacher for the first two distillation rounds; and (3) execute the full pipeline on a single consumer GPU with $\leq$8 GB VRAM. Table 15 evaluates the final empirical results against each target.

Table 15: Assessment of SMART objectives against empirical results (`run_20260609_105305`).

Objective	**Target**	**Achieved**	**Status**
$4\times$ speedup at $>70\%$ quality	$4\times$, $>70\%$	$3.63\times$ (32-step), 75.93% pass rate	**Exceeded**
PPL within $1.5\times$ of Teacher	< 19.50	12.97–15.93 (Rounds 1–2)	**Exceeded**
Run on RTX 2070 Super 8 GB	Yes	Completed 7 rounds, zero OOM errors	**Achieved**

The **32-step distilled Student** is the optimal operating point: it is $3.63\times$ faster than the 128-step Teacher, achieves the highest Promptfoo score of the entire study (75.93%), and maintains a perplexity of 15.93—well inside the $1.5\times$ quality envelope. The **64-step distilled Student** is equally notable because it matches the 128-step Teacher in pass rate while running $1.94\times$ faster, definitively proving that distillation outperforms naïve step reduction: the 64-step non-distilled control scored only 70.37%.

The **8-step Student** pushes acceleration to $12.25\times$ but drops the Promptfoo pass rate to 62.96% and suffers from the qualitative failure modes documented in Chapter 6. It therefore meets the acceleration target but falls short of the quality-retention threshold, defining the practical lower bound of usable distillation for this architecture on this hardware.

Engineering significance of the 8 GB achievement. Standard reproductions of LLaDA-8B distillation reported in the literature typically assume A100 or H100 GPUs with 40–80 GB VRAM. By implementing iterative disk-swapping, 4-bit NF4 quantization, 8-bit AdamW, and gradient checkpointing, this thesis demonstrates that identical scientific outcomes—recursive teacher–student halving down to a single step—are achievable on a **$450 consumer card** released in 2019. The total pipeline wall-clock time for seven rounds was approximately **3.5 hours**, of which evaluation dominated. This accessibility result is arguably as important as the algorithmic result, because it lowers the barrier to entry for researchers without datacenter access.

7.2 Future Research Directions

While the pipeline is fully functional and the empirical objectives are met, several limitations surfaced during validation that motivate concrete avenues for future research.

1. Eliminating the disk-swapping latency bottleneck. The iterative caching mechanism is memory-safe but I/O-bound, spending 400–900 seconds per round writing and reading `pickle+gzip` trajectories. Future work could replace this format with memory-mapped Safetensors or PyTorch TensorStore, enabling zero-copy reads and faster deserialization. On systems with 12–16 GB VRAM, overlapping Teacher caching and Student training in a producer–consumer pattern could eliminate disk I/O entirely.

2. Scaling LoRA capacity on low-VRAM budgets. The current configuration targets only `q_proj` and `v_proj`. Future iterations could add `k_proj` and `o_proj` to the target module set, doubling trainable parameters while remaining within the 8-bit optimizer budget, or explore DoRA (Weight-Decomposed Low-Rank Adaptation), which has demonstrated improved parameter efficiency over vanilla LoRA at equivalent rank.

3. Hybrid and curriculum-aware loss functions. The current loss is pure KL divergence over the full vocabulary. Future work could augment this with a cross-entropy term on ground-truth completions, anchoring the Student to real text rather than purely to the Teacher's soft labels, or with a contrastive consistency loss that penalizes argmax divergence at the same diffusion midpoint. A perplexity-aware curriculum that dynamically adjusts `target_step` within each round based on the Student's current perplexity would further adapt the training to individual model progress.

4. Temperature annealing and stochastic decoding. The fixed $\tau = 2.0$ was sub-optimal for extreme-compression rounds ($\leq$8 steps). A round-dependent temperature schedule—either increasing to accommodate entropy collapse or decreasing to sharpen distributions as step counts fall—could substantially mitigate the mode-averaging artifacts documented in Chapter 6. Whether `remasking='random'`

during Teacher caching produces more robust interpolation points at very low step counts relative to the current `low_confidence` strategy warrants systematic evaluation.

5. Scaling to larger base models and longer contexts. The pipeline was validated on LLaDA-8B with `gen_length=64`. Testing with generation lengths of 128 and 256 tokens would reveal whether the error-accumulation inflection point between 16 and 8 steps shifts as the sequence length increases. Multi-GPU pipeline parallelism—where one GPU holds the Teacher and another the Student—could eliminate disk swapping entirely while still fitting within the 8 GB per-card budget common in consumer workstations.

6. Rigorous statistical validation. The current evaluation uses 12 prompts and 50 training examples due to time constraints on a single local machine. A follow-up study should expand the Promptfoo suite to 100+ prompts spanning multiple languages and reasoning types, report bootstrap confidence intervals on perplexity and pass-rate metrics, and supplement LLM-as-a-Judge evaluation with human assessment to calibrate the Llama 3.1 8B judge's alignment with human preferences.

These directions, taken together, form a credible roadmap for transforming the current proof-of-concept into a production-grade tool for accelerating discrete diffusion language models on commodity hardware—one that would make the benefits of advanced parallel text generation accessible to the broadest possible research and engineering community.

References

[1] S. Nie, D. Zhang, and X. Liu, "Large language diffusion models (llada): A new paradigm for parallel generation," *arXiv preprint arXiv:2502.09992*, 2025.

[2] T. Salimans and J. Ho, "Progressive distillation for fast sampling of diffusion models," *ICLR*, 2022.

[3] Y. Song and P. Dhariwal, "Consistency models," *ICML*, 2023.

[4] D. Zhang and Y. Li, "A comprehensive survey on diffusion language models: 2024-2026," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2026.

[5] X. Li, Y. Wang, et al., "Accelerating diffusion language models: A survey on sampling and post-training methods," *arXiv preprint arXiv:2406.10998*, 2024.

[6] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," *Advances in Neural Information Processing Systems*, vol. 33, pp. 6840–6851, 2020.

[7] J. Song, C. Meng, and S. Ermon, "Denoising diffusion implicit models," *ICLR*, 2021.

[8] S. Sahoo et al., "Masked diffusion language models," in *NeurIPS*, 2024.

[9] A. Vaswani et al., "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.

[10] J. Su et al., "Roformer: Enhanced transformer with rotary position embedding," *Neurocomputing*, vol. 568, 2024.

[11] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, 2015.

[12] E. J. Hu et al., "LoRA: Low-rank adaptation of large language models," *ICLR*, 2022.

[13] Promptfoo, *Promptfoo: Open-source llm testing and evaluation*, `https://www.promptfoo.dev`, 2024.

[14] E. Strubell, A. Ganesh, and A. McCallum, "Energy and policy considerations for deep learning in nlp," University of Massachusetts Amherst, Tech. Rep., 2019.